Toward Self-Improving Robotic Agents

From Interaction Experience to Reusable Manipulation Capabilities

Chenyi Zi[1,2,*], Shuaiqi Cheng[1,2,*], Wenxuan Song[2], Jiayi Chen[2], Yuheng Ji[3], Lvchen Dai[1,4], Shan Peng[2,5], Chengliang Li[2], Zongyang Qiu[2], Haiying Xu[2], Yujia Bai[2], Zheng Zhang[6], Haoang Li[2], Jia Li[1,2,†]

[1]ALLE AI Lab
[2]The Hong Kong University of Science and Technology (Guangzhou)
[3]Institute of Automation, Chinese Academy of Sciences
[4]The University of Western Australia [5]Xiwei Technology (Guangzhou) Co., Ltd.
[6]Nanyang Technological University

[*]Equal contribution [†]Corresponding author

Abstract

Recent advances in embodied artificial intelligence have enabled robot manipulation agents to transition from executing static policies to autonomously constructing reusable skills, programs, and behavioral artifacts. However, successful execution, retained utility, and repeatable acquisition support fundamentally different empirical claims. This mechanism-oriented analysis critically examines 162 purposively selected primary studies through two complementary questions: how agent decisions reach robot control, and how capabilities are acquired and retained. We connect reported gains to their underlying artifacts, recipients, comparators, and evaluation boundaries. Retained programs and guidance can improve skill selection, command formulation, and recovery without altering the underlying actor, whereas neural adaptation and retention mechanisms govern distinct system components. However, aggregate performance gains can mask condition-specific losses or complete task failures. Applying our proposed five-level capability framework reveals that while intermediate levels of modular and simulation-based retention are widespread, no inspected system satisfies the Level-5 criterion of autonomous physical self-improvement under rigorous human- and procedural-intervention accounting. Although repeatable acquisition, transfer, and retention co-occur in bounded simulation under stated interaction budgets, independently verified hardware self-improvement with connected transfer, retention, and full physical cost accounting remains unresolved. These findings motivate standardized evaluation protocols that explicitly couple retained capability gains with repeatable acquisition and rigorous intervention-cost accounting.

Keywords: robot manipulation; robotic agents; capability acquisition; self-improvement; continual learning; evaluation standards.

Contents

1 Introduction **3**

2 Problem Formulation and Analytical Framework **5**
2.1 Execution and Capability Acquisition . . . 5
2.2 Update Targets and Their Effects on Execution . . . 6
2.3 Acquisition Processes, Capability Lineage, and Agency Boundaries . . . 6
2.4 Evidence Criteria and Claim Boundaries . . . 7
2.5 Review Scope and Coding Protocol . . . 8

3 Agent-Mediated Manipulation: Grounding, Coordination, and Control **8**
3.1 Physical Grounding and Ephemeral Belief Dynamics . . . 8
3.2 Task Planning, Decomposition, and Multi-Agent Coordination . . . 9
3.3 Skill Selection, Composition, and Policy Handoffs . . . 10
3.4 Decision Interfaces for Physical Control . . . 11
3.5 Execution Monitoring and the Operational Limits of Track A . . . 13

4 Capability Acquisition Targets: Artifacts and Ingestion Contracts **15**
4.1 Declarative Memory and Relational Schemas (M_k) . . . 15
4.2 Executable Skill Repositories and Policy Graphs (S_k) . . . 16
4.3 Orchestration Harnesses, Tool APIs, and Context Builders (H_k, T_k) . . . 17
4.4 Neural Parameters: Policies, Representations, and Evaluators (θ_k) . . . 18
4.5 Environmental Assets and Learning Procedures (E_k) . . . 19
4.6 Cross-Target Ingestion Realities and the RSI Case Analysis . . . 20

5 Capability Revision, Testing, and Retention **21**
5.1 Experience Sourcing and Active Exploration . . . 21
5.2 Failure Diagnosis and Update Attribution . . . 22
5.3 Candidate Construction . . . 23
5.4 Promotion, Regression Checks, and Rollback . . . 24
5.5 Supervision and Resource Budgets . . . 25

6 Empirical Realities: Capability Lineage and the Evidence Ledger **26**
6.1 Evaluation Designs and Attribution Boundaries . . . 26
6.2 Evidence Levels in the Reviewed Corpus . . . 27
6.3 Diagnostic Case Analysis and Attribution Boundaries . . . 29
6.4 Cross-Mechanism Empirical Synthesis . . . 29
6.5 Repeated Acquisition, Transfer, and Retention . . . 31
6.6 Observed Losses and Unresolved Evaluation Boundaries . . . 32

7 Discussion and Future Directions **36**
7.1 Methodological Reflections: Three Systemic Dilemmas . . . 36
7.2 Future Directions: Reliable and Sustained Capability Acquisition . . . 36
7.3 Limitations of the Review . . . 37

8 Conclusion **37**

References **37**

1. Introduction

Robot manipulation is expanding beyond static, policy-centric execution toward agent-mediated capability development [195]. While modern Vision–Language–Action (VLA) architectures achieve broad semantic generalization across instructions and objects [7, 8, 43, 59, 206], what the robot encounters during deployment (failed grasps, recovered errors, newly observed object properties) is rarely retained in a form that later tasks can reuse. To address this limitation, agent-mediated frameworks coordinate task decomposition, tool use, and verification around physical execution [45, 50, 78]. Crucially, their role now extends beyond invoking pre-existing behaviors to constructing and revising capabilities themselves—synthesizing control programs, collecting physical experience, and organizing learning experiments [14, 165]. Since sustained deployment needs such experience to accumulate rather than be repaired by hand, we ask: *How do robot agents convert physical interaction experience into reusable manipulation capabilities for subsequent tasks?*

Answering this question requires connecting what computational artifact is updated with how that update alters physical execution. System updates address distinct bottlenecks across the autonomy stack: persistent memory supplies task constraints or guides skill selection; programs structure multi-step control flow and error recovery; harness revisions alter observation filtering and API dispatch; and neural updates adapt motor actuation or spatial representations. Learning resources act indirectly by shaping optimization for subsequent policies. Crucially, these disparate mechanisms can produce identical gains in benchmark success: improvements may reflect more appropriate skill selection, better routine coordination, or more compliant motor execution. An update taxonomy alone does not show how a change reaches behavior, and an execution taxonomy alone does not show how capabilities accumulate. Explaining capability development requires jointly investigating three coupled questions: *what changes, how subsequent execution ingests that change, and which dimension of manipulation behavior improves*.

This conversion is challenging because physical outcomes depend on tightly coupled perceptual, planning, and control loops. A failed grasp may originate in calibration drift, kinematic misplanning, or unmodeled friction, making the true update target uncertain: diagnosis must causally link observed failures to a modifiable component that can alter behavior. Even when an update is successfully synthesized, localized repairs frequently fail complete tasks: an approach trajectory can satisfy reachability checks yet leave the workspace in a state that renders subsequent grasping infeasible [150]. Moreover, capability updates risk catastrophic regression on previously mastered tasks [36, 57]. Evaluating such updates therefore requires keeping two questions apart. The first concerns the *update artifact*: whether it improves complete-task outcomes, transfers to new tasks, and preserves earlier capabilities. The second concerns the *acquisition process*: whether independent runs repeatedly produce such effective updates under explicitly stated human-intervention and reset costs.

Prior work examines this space along separate axes: VLA architectures and sensorimotor learning [102], functional components of embodied agents [120], context-conditioned robot learning [42], and update mechanisms in self-evolving digital agents [3]. These lines of work treat models, execution, and updates in depth, but largely separately. Less systematically connected is the full chain from interaction experience, through the update it produces and how subsequent execution uses that update, to the resulting physical effect. Table 1 summarizes these perspectives and the connecting question pursued here.

Our framework couples **agent-mediated execution (Track A)**, which coordinates physical behavior under a fixed capability configuration, with **capability acquisition (Track B)**, which constructs or revises that configuration across deployment cycles (Figure 1). We trace this conversion through a *capability lineage*, linking originating experience, the acquisition mechanism, the updated component, its downstream invocation, and the resulting physical performance. By systematically auditing 162 primary manipulation studies across hardware and embodied simulation through October 2026, this review delivers three findings, which address in turn which component updates

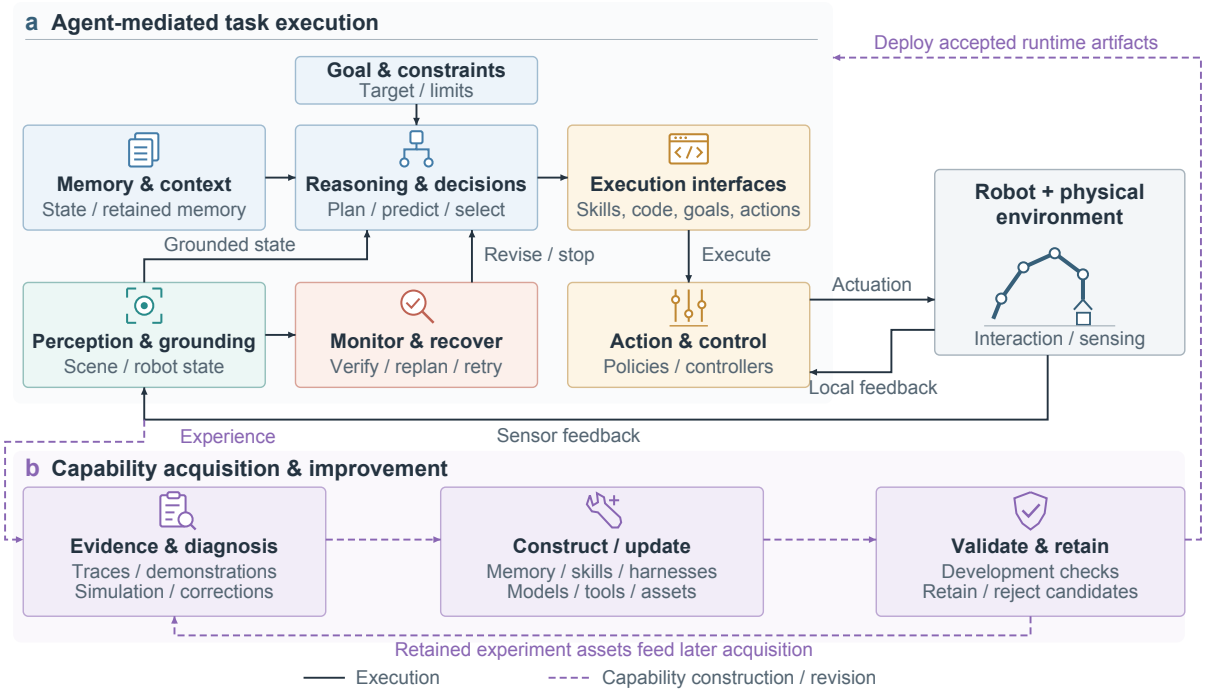


Figure 1. Decoupled architecture of agent-mediated execution and capability acquisition. (a) Task Execution: An embodied agent coordinates grounding, tool use, and closed-loop control under a frozen configuration C_0. Solid lines denote forward actuation and sensorimotor feedback; transient state z_t remains strictly decoupled from persistent memory. **(b) Capability Acquisition:** Interaction traces and external supervision guide candidate synthesis, validation gating, and artifact retention across configurations C_k. Accepted runtime artifacts enter deployment, while developmental assets feed subsequent learning cycles.

yield gains, whether local updates improve complete tasks, and whether the acquisition process repeatedly produces effective updates:

1. **Capability gains are not limited to action-policy updates.** Reported manipulation gains also arise from refined prompt guidance, skill selection, or harness routing while the neural actor remains fixed. Which component is updated therefore matters: locating whether an execution bottleneck lies in semantic reasoning, tool composition, or continuous control should precede the choice of update target.

2. **Locally accepted updates do not guarantee complete-task improvement.** Accumulated skills and locally accepted repairs can fail once composed into complete tasks: a reusable capability needs verified entry conditions and must leave physical states that support downstream skill handoffs. Whether an update artifact is effective must therefore be judged by complete-task outcomes, transfer, and retention of earlier capabilities, not by isolated component metrics alone.

3. **Effective updates are documented, but repeatable acquisition on hardware is not yet established.** Repeated acquisition, transfer, and retention are jointly demonstrated in bounded simulation streams. On hardware, no examined system shows that independent acquisition runs repeatedly produce effective updates with connected transfer, retention, and explicit accounting of human intervention and reset costs.

Section 2 formalizes our analytical framework, system state tuple, and evidentiary protocol. Sections 3–5 examine execution interfaces, update targets, and acquisition processes; Section 6 synthesizes empirical outcomes; Sections 7–8 outline open challenges and conclude.

Table 1. Related perspectives and the connections developed in this survey. The final column identifies questions motivating our synthesis, rather than asserting that prior work omits the corresponding topics.

Related Perspective	Organizing Focus	Connecting Question Pursued Here
VLA Models [102]	Model architectures, sensorimotor learning, and evaluation.	How do updates to surrounding components improve manipulation when the action policy remains fixed?
Embodied Agents [120]	Functional components and model integration.	How does execution experience change the capabilities available for subsequent tasks?
Robot ICL [42]	Context-to-action interfaces and experience-based conditioning.	How do guidance, code, harness, and neural updates enter later execution through different mechanisms?
Self-Evolving Agents [3]	Update targets, mechanisms, and evaluation.	How do physical execution constraints determine whether an update improves the complete task?
Digital Colleagues [192]	Persistent workspaces, reusable procedures, and task completion.	What physical state must be re-observed or restored before a retained procedure can be reused?
Agentic Ethics [116]	Philosophical agency, safety, and governance.	Which acquisition decisions and physical interventions remain dependent on people?
This Survey	**Experience-to-capability conversion through coupled execution and acquisition.**	**What changes, how does later execution use it, and under what conditions does manipulation improve?**

2. Problem Formulation and Analytical Framework

This section formalizes the core theoretical substrate connecting physical interaction experience, retained computational updates, and subsequent manipulation performance. We first decouple intra-episode execution from cross-episode capability acquisition, formulate the persistent system configuration tuple, and establish the operational lifecycle governing candidate generation and validation gating. Finally, we formalize the capability lineage and evidentiary audit criteria used to assess claims across the surveyed literature.

2.1 Execution and Capability Acquisition

We formalize robot manipulation across two asynchronous timescales: intra-episode closed-loop control governed by continuous sensorimotor transitions, and cross-episode capability acquisition governed by structural configuration mutations. At execution step $t \in [0, T]$, an underlying policy maps observation–action history $h_t = (o_0, a_0, \ldots, o_t)$ and instruction ℓ to an immediate motor action or trajectory chunk:

$$a_t \sim \pi_\theta(\cdot \mid h_t, \ell). \tag{1}$$

Even under fixed weights θ, recurrent hidden states, feature caching, and reactive feedback loops dynamically modulate control within an episode. Such responsiveness reflects runtime execution resilience rather than retained capability acquisition.

An agentic manipulation system coordinates intermediate deliberative decisions, including task decomposition, tool selection, spatial constraint formulation, and closed-loop verification:

$$\begin{aligned} (u_t, z_{t+1}) &= \mathcal{A}_{C_k}(o_t, \ell, z_t, F_t), \\ a_t &\sim \pi_{\text{actor},C_k}(\cdot \mid u_t, o_t), \end{aligned} \tag{2}$$

where u_t is an intermediate executable decision (such as a parameterized skill invocation, programmatic subroutine, or spatial objective); z_t designates transient belief; F_t represents physical execution feedback; and configuration C_k (defined below) parameterizes the available reasoning, orchestration, and action components. The continuous action chunk a_t is realized by the underlying actor π_{actor} conditioned on u_t and immediate observation o_t. This functional decomposition applies equally to unified foundation models and modular harnesses. Agency is defined not by the presence of a language model, but by the structural orchestration of intermediate decisions, tool endpoints, and physical feedback loops.

To evaluate capability retention across extended deployments, we define the *physical execution context* as the temporal state space governing robotic continuity: task progress, timestamped 3D scene estimates, kinematic configurations, capability registries, and execution traces. Crucially, retaining symbolic software state (such as an object prehension flag or estimated bounding box) preserves an internal belief, but guarantees neither physical validity after unobserved slippage nor autonomous workspace recovery. Physical continuity demands renewed sensorimotor verification; maintaining software state and acquiring a transferable manipulation capability represent fundamentally disparate claims.

2.2 Update Targets and Their Effects on Execution

Across successive deployment cycles ($k \in \mathbb{N}$), the system's persistent operational capacity is parameterized by the architectural configuration tuple:

$$C_k = (\theta_k, H_k, M_k, S_k, T_k, E_k), \tag{3}$$

where θ_k contains trainable neural parameters; H_k designates the execution harness and prompt orchestrator; M_k stores declarative knowledge and textual guidance; S_k contains callable skills or programs; T_k specifies callable tool contracts; and E_k records developmental assets including simulation environments, rewards, and training recipes. The first five components govern physical execution, whereas E_k guides downstream learning or evaluation. Artifacts are classified strictly by representation and ingestion: textual planner guidance belongs to M_k, executable code to S_k, and context orchestration to H_k, regardless of source-paper terminology.

The distinction matters because these targets enter execution through fundamentally different routes: retrieved M_k influences constraints, command formulation, or skill selection; invoked S_k supplies reusable behaviors and their composition; revisions to H_k or T_k change context construction, dispatch, monitoring, or recovery; updates to θ_k modify continuous control, reasoning, or verification; and changes to E_k act through subsequent learning cycles. As synthesized in Figure 2, our framework establishes two orthogonal classification axes: the **execution interface** describes how decisions reach physical control, while the **update target** identifies which component of C_k changes.

2.3 Acquisition Processes, Capability Lineage, and Agency Boundaries

Candidate capability generation and deployment promotion are governed by:

$$\widetilde{C}_{k+1} = \mathcal{U}(C_k, \tau_k, F_k; B_k), \qquad C_{k+1} = \mathcal{G}(C_k, \widetilde{C}_{k+1}; \mathcal{V}, B_k), \tag{4}$$

where $\mathcal{U}$ constructs candidate mutation $\widetilde{C}_{k+1}$ from interaction traces τ_k and feedback F_k under resource budget B_k, and promotion gate $\mathcal{G}$ accepts or rejects it using validation evidence $\mathcal{V}$ from development partition $\mathcal{D}_{\text{val}}$. Writing this formalization accommodates both automated pre-deployment validation suites and systems lacking explicit gates (where $\mathcal{G}$ acts as a direct adoption rule with $\mathcal{V} = \emptyset$).

Our analytical unit, a *capability lineage*, links originating experience or supervision, the diagnosis and construction mechanism, the changed component, the adoption decision, the later-use interface, and the comparative physical outcome. Tracing where an update is retrieved, invoked, or learned connects the acquisition process to a testable behavioral effect.

Decision responsibility across Equation 4 is recorded separately from artifact utility. We formalize four operational boundaries of agency: *Researcher-selected*, where humans curate data, objectives, and promotion rules; *Teacher-assisted*, where external expert demonstrations or dialogue guide acquisition; *Delegated agent improvement*, where an agent proposes and tests updates within a static development workflow; and *Agent-selected improvement*, where the agent autonomously directs diagnosis, exploratory trials, and promotion gates. These categories delineate operational decision authority rather than a hierarchy of measured artifact utility.

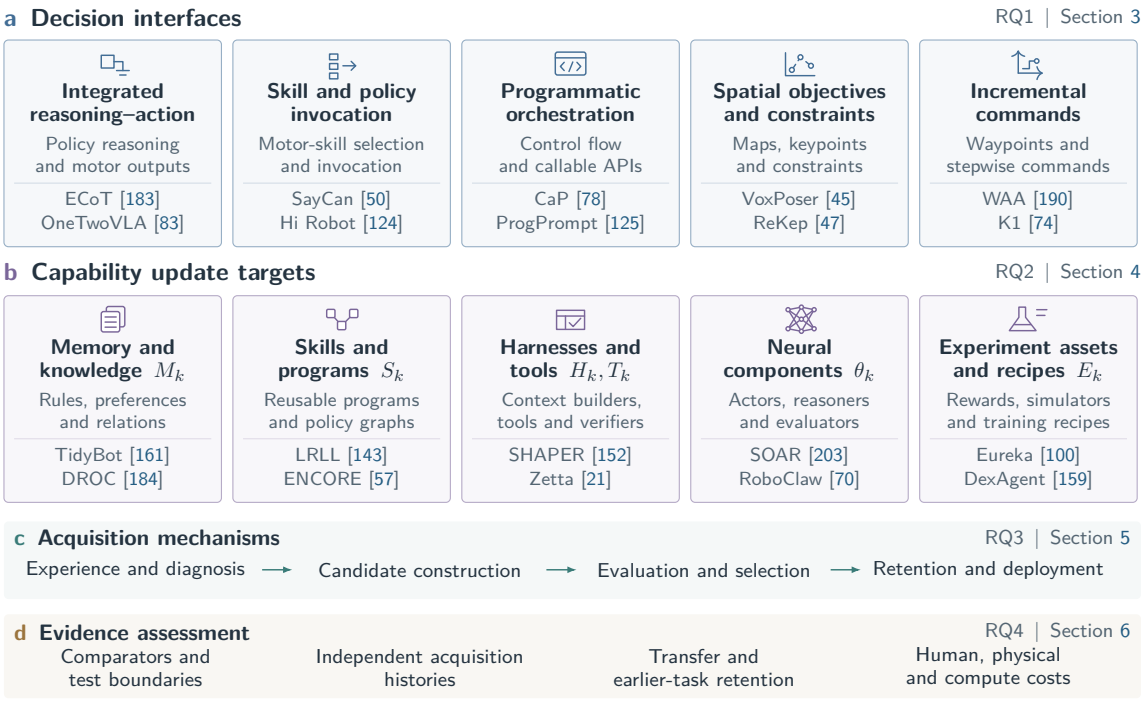


Figure 2. Analytical map of agent-mediated manipulation and capability acquisition. The two classification bands distinguish decision interfacesfrom update targets, followed by acquisition mechanisms and evidence assessment.

2.4 Evidence Criteria and Claim Boundaries

A lineage describes how a change could affect execution; evidence determines which outcomes are established. We distinguish artifact construction, persistence, later use, and comparative benefit. Transfer, earlier-task retention or regression, resource costs, and acquisition repeatability further characterize that benefit. Repeated execution of one artifact measures runtime variance, whereas independent acquisition histories test whether the procedure reliably produces useful artifacts. Our framework formalizes five progressive evidence levels (Table 4):

L1: Execution baseline. Agent-mediated manipulation is documented without established construction of a capability artifact.

L2: Capability construction. An artifact is constructed or adapted, but persistence and subsequent execution are not jointly verified.

L3: Persistence and later use. The constructed artifact survives episode termination and is actively loaded or invoked by subsequent execution or learning workflows.

L4: Measured manipulation gain. A connected artifact lineage supports reported physical gain over a frozen comparator under source-declared separation of acquisition or selection from evaluation.

L5: Repeatable physical self-improvement. Independent acquisition histories yield protected-test gains, with cross-task transfer, earlier-task retention, and low human dependence jointly supported under declared intervention criteria and accounting periods.

These levels describe evidence in the inspected corpus, not a universal ranking of system quality. L5 is a review-defined joint criterion; failure to meet it does not negate supported components. Each record receives the highest level supported by one connected experimental chain. Simulation (S), hardware (H), and joint evidence (B) identify the credited endpoint. Offline training and teacher assistance remain eligible for L4 when evaluation separation holds.

2.5 Review Scope and Coding Protocol

Four progressive questions organize the synthesis:

RQ1: Execution. Through which interfaces do available capabilities produce physical manipulation, and where does sensory feedback alter decisions?

RQ2: Update targets. Which computational components are constructed or revised, and how are they ingested by subsequent execution?

RQ3: Acquisition. How do interaction experience and supervision drive update generation, validation, and adoption?

RQ4: Evidence. What empirical evidence substantiates subsequent manipulation gains, transfer, retention, and repeatable acquisition under bounded intervention?

The audited corpus comprises 162 primary manipulation studies available through October 2026 across hardware and embodied simulation. Eligible studies investigate goal-directed physical manipulation with inspectable execution or acquisition mechanisms and reported evaluation outcomes. Purely digital software agents, standalone perception modules, navigation-only systems, and ungrounded dialogue are excluded.
For each primary study, our structured ledger cross-verifies seven core dimensions: execution interface, mutated configuration target, persistence and ingestion, comparator baseline and reported gain, acquisition/test separation, acquisition repeatability, and human/reset intervention budgets. Unreported properties are explicitly recorded as *unresolved evidence* rather than assumed to be absent, preventing negative inferences from missing reporting. Extraction is AI-assisted and cross-validated through deterministic consistency checks, with complete query strings, screening dispositions, and source audit records preserved in the accompanying ledger. Section 6 presents the resulting empirical synthesis without pooling incompatible success metrics.

3. Agent-Mediated Manipulation: Grounding, Coordination, and Control

An execution-stage embodied agent translates high-level task specifications into continuous physical actions by grounding semantic goals into metric spatial frames, coordinating callable actors, and leveraging real-time feedback to arbitrate corrective interventions. Under the execution boundary formalized in Section 2.3, this section addresses **RQ1**: how agent decisions interface with continuous robot control, which observation structures and intervention points support error recovery, and what operational limitations arise under a given capability configuration C_k. This analysis holds the configuration fixed during the examined execution, without excluding acquisition at other stages.

3.1 Physical Grounding and Ephemeral Belief Dynamics

Task grounding projects abstract semantic directives onto concrete physical states: target entities, continuous affordance manifolds, spatial coordinate frames, and contact prerequisites. Representations optimized strictly for semantic object categorization frequently prove inadequate for contact stability, compliance regulation, or force-closure grasp verification. Underlying VLA actors can learn visual grounding priors through target-region reconstruction [130] or alignment with pretrained 3D representations [68]. AffordanceVLA [181] further grounds action generation through object selection, interaction localization, and 3D affordance reasoning. A3D [80] adapts support and stabilization locations from interaction feedback during dual-arm assembly.

Scene Continuity and State Tracking Under Dynamic Contact. Execution harnesses diverge substantially in how they maintain spatial continuity across contact events, frictional slip, and

visual occlusions. As established in Section 2.1, a stored prehension flag or 3D bounding volume reflects historical evidence; physical disturbances can invalidate this internal state without generating an inspectable software exception.

Implementations span two distinct tracking paradigms. In ephemeral session tracking, explicit task progress indicators and coordinate frames exist exclusively within the temporal lifecycle of a single trial, re-initializing the internal world model upon episode termination, as in SimpleARM [194]. Conversely, spatio-temporal world-state representations track physical entities across extended interactions: RoboStream [48] preserves object identities and action-triggered transitions across continuous visual occlusions, while MessyMem [5] organizes dynamic 3D scene graphs across long-horizon executions.

Multi-step state tracking maintains the agent's task belief z_t. If this state is reset without retaining a component for subsequent use, its within-task persistence does not establish cross-task acquisition. Whether a record contributes to later capability depends on its lifetime and use, as defined in Section 2.1. In contrast, learned world representations can encode reusable priors over state transitions [158]. The decisive control-theoretic distinction is whether an internal memory entry denotes raw sensory evidence, an inferred geometric hypothesis, or an actively confirmed physical contact state. Following external preemption or visual occlusion, downstream action dispatch demands renewed physical verification that prerequisite states remain intact.

Active Sensing, Probing, and Epistemic Uncertainty Reduction. When passive sensory observations leave the state distribution uncalibrated, an embodied agent can actively interrogate the physical environment prior to irreversible actuation. In physical probing, the manipulator executes deliberate, low-energy exploratory motions to infer latent physical parameters: PhysCaP [82] evaluates epistemic uncertainty and commands executable probes to estimate mass and surface stiffness from proprioceptive feedback before executing dynamic tasks. In active viewpoint selection, the agent repositions external or wrist cameras to resolve occlusions: VAP-TAMP [109] couples precondition and effect monitors to camera movements when existing visual perspectives are insufficient for geometric predicate evaluation. In clarification interactions, the system defers action commitment to human supervisors: KNOWNO [117] uses conformal prediction sets over candidate plans, triggering targeted human queries whenever the prediction set contains multiple competing hypotheses. Each exploratory or interactive modality expends physical execution time, joint actuation budgets, or human oversight; their systemic utility must therefore be audited against the operational costs of active probing trajectories, sensor displacement, and human intervention.

3.2 Task Planning, Decomposition, and Multi-Agent Coordination

Task planning maps grounded manipulation goals into sequential subgoals or conditional programs, drawing on semantic and spatial reasoning [51, 139]. The operational trade-offs governing deliberative planning center on how alternative representations enforce physical kinematic feasibility, when sensory feedback triggers online replanning, and how multi-embodiment architectures arbitrate shared workspaces. RationalVLA [129] adds a prior check of instruction admissibility, rejecting defective commands before manipulation.

Representations for Manipulation Decomposition. Across current architectures, three primary planning representations impose fundamentally different structural constraints on physical execution:

- *Affordance-Grounded Value Ranking:* Semantic action proposals are filtered through learned physical feasibility scores. As pioneered by SayCan [50], an autoregressive language model scores task relevance while a learned value function evaluates the real-time physical feasibility of candidate skills. This approach grounds high-level selection without requiring

analytical dynamic world models; however, it remains strictly bounded by the static repertoire of pre-trained skills and greedily evaluates skills step by step rather than searching over continuous geometric state manifolds.

- *Symbolic Action Schemas and Predicate TAMP:* Task structures are explicitly formalized via rigorous preconditions, continuous parameters, and expected physical transitions. PLanAR [32] enforces formal object predicates and action schemas with observation-based effect verification, whereas SayPlan [115] navigates hierarchical 3D scene graphs and refines proposed plans using scene-graph simulator feedback prior to physical actuation. These formalisms provide verifiable task progression, yet their physical execution fidelity remains vulnerable to grounding noise in continuous state estimation.
- *Programmatic Logic and Executable Assertions:* Task plans can be represented as structured programs with conditional branches, loops, and explicit execution checks. ProgPrompt [125] generates situated task plans with assertions and recovery routines, while RoboScript [13] translates free-form manipulation instructions into executable scripts using geometric planning and control modules. Such representations make task-level control flow explicit and inspectable, although their effectiveness depends on the available robot APIs and execution feedback.

Replanning Triggers and Closed-Loop Preemption. Replanning is triggered when empirical feedback violates formal assumptions: an unfulfilled precondition, an unachieved postcondition despite nominal controller termination, or an unreachable kinematic waypoint. Under the Track A paradigm, these events dynamically restructure the sequence of callable actions within the active trial; they alter no persistent capability representations.

Multi-Agent Coordination and Workspace Interference. In collaborative manipulation settings, spatial allocation couples subtask dispatch with physical interference constraints. RoCo [103] employs multimodal dialogue rounds between heterogeneous manipulators to negotiate task allocations and 3D waypoints, subsequently dispatching agreed coordinates to centralized motion planners that check spatial collision constraints under the represented geometry. In human-robot co-working contexts, MOSAIC [149] integrates interactive planning with probabilistic human trajectory forecasting, dynamically arbitrating subtask assignments during shared culinary assembly. Furthermore, heterogeneous infrastructures such as ABot-Claw [49] coordinate diverse robotic embodiments across unified visual grounding, mobile navigation, and object handovers. RoboOS-NeXT [137] coordinates heterogeneous robots through shared spatio-temporal-embodiment memory. In all collaborative settings, linguistic consensus remains purely declarative; physical feasibility demands continuous geometric validation within the shared contact envelope.

3.3 Skill Selection, Composition, and Policy Handoffs

Skill composition selects a callable actor, binds its continuous arguments to the active scene geometry, and verifies whether its terminal kinematic state satisfies the entry conditions of subsequent primitives. The execution view treats composition as an orchestration problem over an established repertoire.

Skill Interfaces and Compositional Invariants. Extending procedural abstractions in digital systems [192], physical manipulation skills are governed by five operational boundaries: (i) precondition and feasibility boundaries that verify spatial reachability, collision-free corridors, and prehension states; (ii) scene-grounded bindings that map abstract parameters to 6-DoF grasp poses or tracked surface keypoints; (iii) continuous motion substrates that execute trajectories via feedback controllers, dynamic movement primitives, or learned visuomotor policies; (iv) post-execution

invariant monitors confirming that the targeted physical transition occurred; and (v) interruption thresholds on quantities such as torque and velocity, whose effect on stopping behavior and scene stability depends on the controller and task. Compositional feasibility therefore depends on whether the terminal states produced by skill $\mathcal{S}_i$ satisfy the initiation conditions of skill $\mathcal{S}_{i+1}$.

Delegation to Foundation Actors. Hierarchical architectures frequently partition decision authority between high-level multimodal deliberators and low-level sensorimotor actors. Hi Robot [124] partitions control hierarchically: a high-level multimodal planner processes user intent and camera frames to emit segmented directives to a low-level VLA executing continuous joint control. Conversely, systems like Pigey [26] and VLAs-as-Tools [65] encapsulate pre-trained foundation policies directly as callable tool endpoints within an orchestration harness, utilizing returning observation streams to arbitrate downstream dispatch. Cortex [113] aligns planner directives with low-level actor interfaces via structured skill descriptors and canonical prompt schemas. OpenHelix [18] uses latent conditioning between a vision-language model and a diffusion actor, while VLA^2 [197] supplies OpenVLA with retrieved knowledge and object detections.

Handoff Boundaries Between Heterogeneous Policies. Handoffs across heterogeneous policies (such as transitioning from an analytical trajectory planner to a learned diffusion actor) frequently encounter boundary mismatches in action horizons, coordinate frames, and observation dynamics. RoboHarness [44] explicitly resolves these transition mismatches via a dedicated *Memory Bridge*, retrieving historical trajectories, learning a local progress ranking, and selecting reachable handoff configurations compatible with the incoming actor. Analogously, 2AM [41] decouples temporal memory from motor execution, maintaining long-horizon context within the high-level reasoner while deploying an episodically stateless action generator conditioned on semantic instructions and spatial guidance hints. To arbitrate dynamic transitions between modular analytical planners and contact-rich end-to-end controllers, RoboAware [201] discretizes skill execution into a five-stage semantic schema (P^5) and trains an outcome-aware coordinator via state-locked counterfactual branching, dynamically assigning execution responsibility across policy families at each stage boundary based on anticipated physical returns. These bridging mechanisms help mitigate transition failures; their systemic contribution must be evaluated against unbridged baselines.

3.4 Decision Interfaces for Physical Control

The *decision interface* establishes the structural contract between high-level agent reasoning and physical robot control. Table 2 and Figure 3 compare five canonical execution patterns, delineating their action representations, motion substrates, feedback timing, and runtime intervention points.

Pattern 1: Integrated Reasoning–Action Interfaces. A unified sensorimotor model generates intermediate reasoning traces, subgoal images, or spatial heatmaps directly alongside continuous motor commands. In Embodied Chain-of-Thought (ECoT) [183], the network autoregressively emits textual and spatial bounding tokens prior to outputting discretized action tokens. OneTwoVLA [83] incorporates discrete deliberation tokens that alternate dynamically between linguistic reasoning and continuous chunk generation, integrating error-recovery rollouts directly into the policy training distribution. At the actor-model level, related approaches couple visual prediction with action generation through optical-flow intermediates [200], joint image–action denoising [12], unified world–action modeling [131], affordance-aware joint world–action prediction in AffordanceWAM [179], or cross-embodiment curricula with hybrid temporal attention and joint visual–action prediction as in Kairos [140]. Integrated architectures reduce inter-module translation overhead, but their tightly coupled reasoning and control can make failure attribution and targeted correction less transparent.

Table 2. Classification and Execution Characteristics of the Five Physical Execution Interfaces. Timing is described qualitatively because rates depend on the model, controller, and task. Runtime interventions are distinct from retained updates.

Interface Pattern	Decisive Decision (u_t)	Motion Substrate	Feedback Timing	Runtime Intervention
1. Integrated Reasoning–Action	Internal reasoning tokens, visual subgoals, or latent spatial tokens	Monolithic visuomotor policy emitting continuous action chunks	Observation and action-chunk updates; model-dependent latency	Observation refresh or instruction steering through supported inputs
2. Skill / Policy Invocation	Discrete skill identifier with parameterized continuous arguments	Modular, decoupled actors (specialized VLAs, RL, or primitives)	Skill completion, monitoring, or interruption; variable horizon	Skill re-sequencing, target pose re-grounding, or fallback actor dispatch
3. Programmatic Orchestration	Synthesized Python/DSL scripts containing loops and branch logic	Robot APIs invoking controllers, learned actors, or solvers	Program checks and tool returns; generation latency varies	Conditional branches, retries, or current-task replanning
4. Spatial Constraints & Objectives	Continuous 3D cost volumes, relational keypoints, or energy maps	Numerical trajectory optimizers, QP solvers, and operational IK	Scene updates and solver replanning; solver-dependent latency	Cost function re-weighting, spatial constraint relaxation, or keypoint re-registration
5. Incremental Commands	Cartesian pose deltas ($\Delta\mathbf{x} \in \mathbb{R}^6$) and discrete gripper triggers	Robot motion APIs and underlying controllers	Successive observations and commands; inference and motion delays	Step-size modulation, trajectory filtering, or iterative retry prompting

Pattern 2: Skill and Policy Invocation Interfaces. This interface represents agent decisions as discrete skill identifiers with scene-grounded continuous arguments. VLAs-as-Tools [65] exposes pre-trained robot policies as callable tools, allowing a high-level reasoner to select and parameterize existing motor capabilities. The selected actor executes under its own control loop, while the agent monitors termination conditions and determines subsequent invocations. This interface separates high-level decision-making from continuous control, but its effectiveness depends on the coverage, reliability, and compatibility of the available skills.

Pattern 3: Programmatic Orchestration and Tool Interfaces. In this pattern, the agent generates executable programs that compose perception, planning, and control APIs. Code as Policies [78] uses language-model-generated programs to invoke robotic control functions, while AGP [54] incorporates physical execution feedback to iteratively refine generated policies. Programmatic interfaces make control flow, conditional execution, and recovery logic explicit and inspectable. However, their reliability depends on the capabilities exposed by the underlying APIs, while code generation and revision introduce additional latency.

Pattern 4: Spatial Objectives and Optimization Constraints. The agent translates semantic goals into continuous 3D cost fields, keypoint trajectories, or optimization energy functions. VoxPoser [45] synthesizes language-conditioned 3D value maps from visual observations, defining objectives for model-based trajectory optimizers. ReKep [47] formalizes manipulation tasks as relational 3D keypoint constraints, resolving path and subgoal constraints via sequential quadratic programming. Spatial interfaces delegate trajectory generation to numerical solvers; dynamic stabilization and contact compliance depend on the underlying controller. Related approaches also use explicit geometric inference to translate learned spatial predictions into robot actions [52]. However, the reliability of spatially grounded interfaces remains sensitive to perceptual registration errors, including depth noise, occlusions, and reflective surfaces.

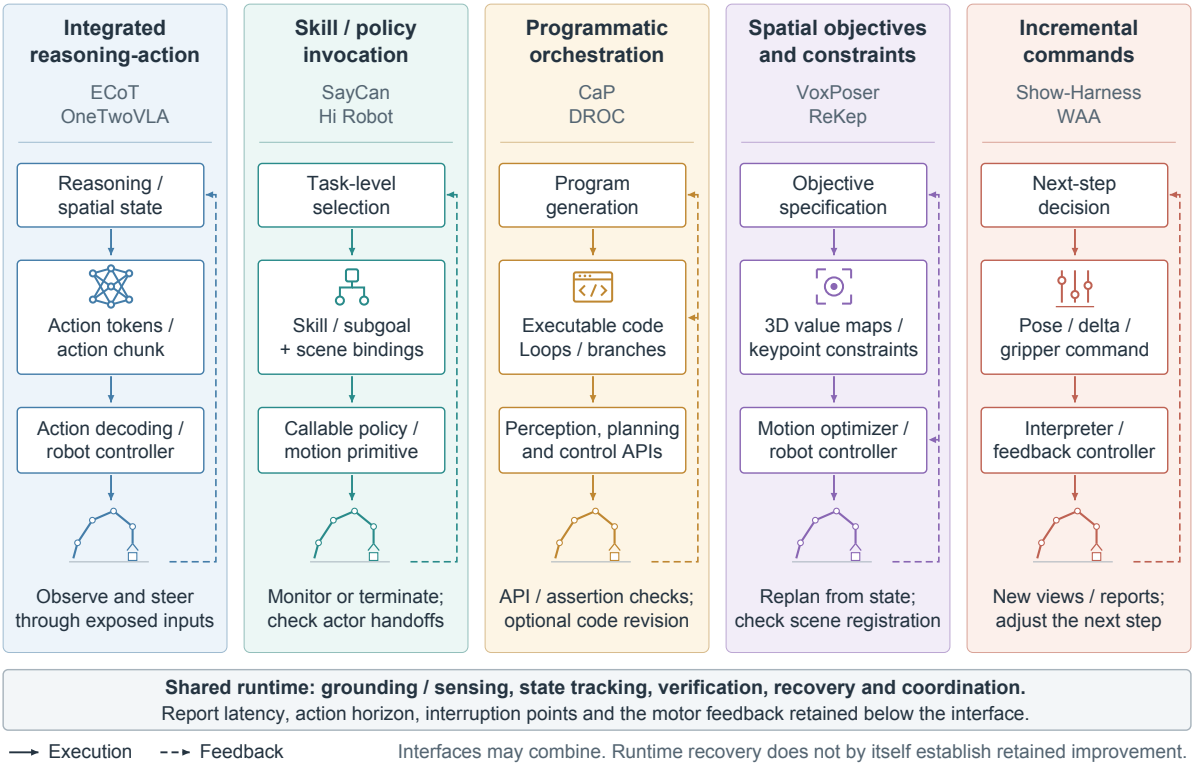


Figure 3. Structural comparison of the five physical execution interfaces. Columns delineate interface patterns from monolithic policies to modular and incremental guidance. Solid arrows trace forward command dispatch from intermediate agent decisions u_t to continuous motion substrates. Dashed arrows mark closed-loop sensory feedback and failure remediation pathways returning to their respective decision boundaries.

Pattern 5: Incremental Command Interfaces. The agent deliberator operates as a low-frequency setpoint corrector, emitting continuous Cartesian pose deltas $(\Delta x, \Delta y, \Delta z, \Delta\theta)$, directional waypoints, or discrete gripper triggers. Systems like Show-Harness [16], RoboDawn [31], and World Action Agent (WAA) [190] expose incremental control interfaces to vision models for successive corrective steps. Incremental commands offer frequent opportunities for trajectory correction, but recurrent model inference introduces non-negligible latency. Contact stability depends on the underlying controller and physical interaction between successive setpoints; the command interface alone does not specify an impedance-control implementation.

Timescales and the Feedback Latency Hierarchy. A rigorous claim of closed-loop manipulation must identify which specific loop is closed: low-level joint stabilization, visuomotor chunk generation, or high-level semantic replanning. Their rates depend on the implementation and task. These control loops can operate at different temporal scales. Contact disturbances faster than the high-level feedback cycle require a response from the underlying controller. PD-VLA [128] reduces chunk-decoding latency through training-free parallel fixed-point iterations, addressing one component of sensing-to-action delay. Reporting standards must explicitly state sensing-to-action latency, execution chunk duration, preemption thresholds, and the exact compliance controller governing contact between high-level deliberation cycles.

3.5 Execution Monitoring and the Operational Limits of Track A

Execution monitoring supports state verification, task progress assessment, and reactive recovery under a fixed capability configuration C_0.

Sources of Verification Evidence. Runtime verification relies on four main sources:

- *Privileged Simulation Predicates:* Physics engine states, joint contact manifolds, and collision meshes are directly queried. However, privileged predicates are generally unavailable on physical hardware without equivalent sensing and state estimation.
- *Learned Visual Verifiers:* Multimodal foundation models evaluate pre- and post-condition RGB frames, as in Inner Monologue [46] and Agentic Robot [177]. However, vision-based verification remains highly susceptible to perceptual illusions, visual occlusions, and specular reflections where an object appears prehended despite lacking normal contact force.
- *Geometric and Programmatic Invariants:* Continuous mathematical constraints are verified throughout trajectory execution: Code-as-Monitor [202] executes synthesized monitor scripts checking spatio-temporal invariants among tracked geometric primitives, while DoReMi [33] uses multimodal checks to detect execution deviations and trigger recovery.
- *Proprioceptive and Contact-Force Verification:* Gripper motor currents, joint torque differentials, and tactile sensor arrays are monitored during motion. Force feedback provides direct physical evidence of prehension; a nominal controller completion status code alone establishes neither contact stability nor grasp retention.

Task Progress Monitoring. While verification checks whether expected states or transitions have occurred, progress monitoring assesses how a task evolves over time[cite: 14]. GVL [101], Robo-Dopamine [136], and RoboMeter [76] estimate task progress from visual observations, while ViVa [99] and WVM [160] leverage video generation models for progress-related value estimation[cite: 15]. PRM-as-a-Judge [53, 89] further uses progress curves for process-level evaluation and failure diagnosis. Complementing process-level auditing, benchmark suites such as WorldFailBench [196] evaluate whether predictive and generative world models can faithfully diagnose and anticipate downstream policy execution failures. Such progress and failure estimates can reveal stagnation and regression beyond individual state checks, providing complementary signals for execution monitoring and potential recovery.

Feedback-Driven Recovery. Verification failures can trigger corrective interventions, while progress estimates provide complementary signals for identifying stalled execution. Existing recovery mechanisms operate at different levels: Plan-level replanning routes semantic failure diagnostics back to the high-level planner for symbolic re-sequencing (e.g., CoPAL [56]); action-space residual steering adjusts frozen policy proposals through learned local correction offsets, as in CommitFlow [199]; and verifier-triggered prompt modification utilizes offline-trained stage or risk verifiers to re-prompt a frozen VLA (e.g., VLA-Corrector [126]) without modifying underlying neural parameters. Furthermore, SafeHarness [171] arbitrates obstacle-aware routing and contact points, whereas KNOWNO [117] solicits human input when its calibrated confidence sets indicate multi-modal uncertainty.

Limits of Runtime Adaptation. The five execution interfaces expose different decision and feedback boundaries: integrated policies couple reasoning with motor output, skill interfaces schedule reusable actors, programmatic interfaces organize callable APIs, spatial objectives guide numerical solvers, and incremental commands permit stepwise corrections. Across these interfaces, monitoring and replanning can improve within-episode robustness under a fixed capability configuration. These execution mechanisms also provide feedback and intervention points for later capability acquisition.

A fixed configuration does not preclude new task solutions: agents may solve previously unseen tasks by recomposing available skills, adjusting execution parameters, and leveraging sensory

feedback. However, these adaptations do not by themselves establish persistent capability acquisition. When systematic failures arise from missing motor skills, unreliable state estimation, or incompatible execution interfaces, further improvement may require updating persistent components of C_k across episodes. This motivates **Track B (Capability Acquisition)**, examined in Section 4, which connects retained updates to the execution interfaces through which they affect later behavior.

4. Capability Acquisition Targets: Artifacts and Ingestion Contracts

Capability acquisition (**Track B**) constructs or revises components of the retained configuration C_k defined in Equation (3) for subsequent use. It can occur before deployment or interleave with execution; its defining feature is the retained change rather than an episode boundary. This section addresses **RQ2**: which computational artifacts change, how they enter subsequent execution or learning, and which behaviors and failure modes these changes affect (Figure 4). We examine five groups of update targets in C_k, grouping harnesses and tools together, through their functional representations, ingestion contracts, and effects on manipulation.

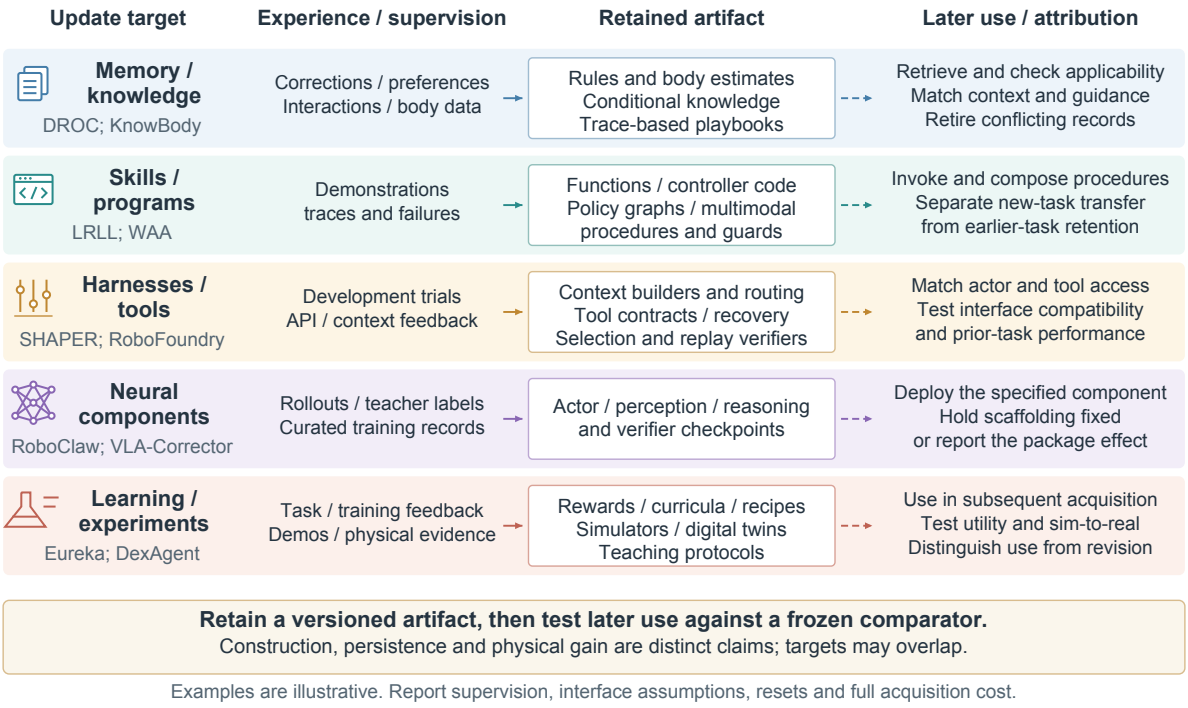


Figure 4. Capability acquisition targets, retained representations, and later-use conditions. The five rows correspond to memory M_k, skills S_k, harnesses and tools H_k, T_k, neural components θ_k, and experiment assets or procedures E_k. Each row summarizes a family of representations rather than the complete implementation of every named example. Solid arrows indicate construction or revision; dashed arrows indicate potential later use.

4.1 Declarative Memory and Relational Schemas (M_k)

Memory updates revise the persistent record of operational guidance $M_k \rightarrow M_{k+1}$ and can influence later decisions without modifying underlying neural parameters θ or executable harness scripts H. Retained rules, body estimates, and examples guide skill selection, parameter binding, or program synthesis. Their utility depends on how records are indexed, whether they remain applicable as physical conditions change, and how relevant information reaches the decision process within its available context budget.

Retained Knowledge Representations. Across the surveyed literature, declarative memory acquisitions populate three primary structural paradigms: *Distilled symbolic rules and preference models* abstract task constraints and user specifications from supplied examples or interaction feedback. TidyBot [161] summarizes user-provided textual placement examples into generalized semantic preference rules across object categories, demonstrating that declarative induction can steer future task executions without weight adaptation. DROC [184] distills language corrections into task-indexed records containing constraints, preferences, object information, and reusable skill parameters, such as gripper poses and pull distances. These records are retrieved using task semantics and object-level visual similarity to guide subsequent planning and code generation. *Queryable body estimates and conditional action-effect schemas* formalize the robot's physical capabilities and morphological constraints. KnowBody [95] maintains an inspectable physical body model alongside condition-dependent action predictors whose applicability is dynamically gated by kinematic body estimates. Candidate body revisions and action-effect schemas are frozen and validated across independent physical trials prior to deployment activation, ensuring that transient tool attachments remain isolated within episodic state. *Procedural playbooks and execution-grounded exemplars* preserve granular behavioral recipes derived from successful execution traces. ARCHITECT [10] distills human corrections and execution-grounded program patches into an expanding exemplar library ingested in full during subsequent program synthesis cycles. Similarly, Recursive Harness Distillation (RHD) [60] extracts intervention heuristics from a frontier teacher model into a structured operational playbook, selectively steering lightweight edge models on when to request verification and when to dispatch autonomous motion primitives.

Epistemic Drift, Invalidation, and Retrieval Trade-Offs. Retained knowledge structures are acutely vulnerable to *epistemic drift*: physical parameters recorded during episode k (such as joint friction estimates, surface contact compliance, or suction seal thresholds) frequently cease to hold following mechanical wear, temperature variations, or payload alterations. Ingestion of uncalibrated memory entries directly injects erroneous inductive biases into downstream reasoning.
RoboHarn-Evo [6] associates knowledge versions with execution evidence labeled as support, opposition, or unverified. Applicable entries are eligible for retrieval when supported without opposing evidence; counter-evidence withdraws the challenged version, while revised content must satisfy the same evidence criterion before reuse. Similarly, Real2Gym [118] extracts reusable procedural packages post-execution to bootstrap subsequent context construction. However, scaling the volume of M_k introduces severe retrieval-capacity tensions: unrestricted retrieval consumes bounded context windows, while selective top-k filtering risks omitting critical geometric invariants or safety preconditions.

4.2 Executable Skill Repositories and Policy Graphs (S_k)

Skill acquisition constructs or refactors callable computational procedures ($S_k \rightarrow S_{k+1}$), encompassing parameterized motion functions, abstract syntax tree (AST) patches, and executable policy graphs. These artifacts preserve explicit control flow and callable operations for later execution. Their nodes or function calls may invoke learned models, planners, or controllers, so retaining a program does not imply deterministic reasoning or eliminate runtime model calls.

Structural Representations of Programmatic Skills. The synthesis of executable manipulation procedures manifests across four architectural representations:

- *Parameterized Functional Primitives and Clustering:* Continuous execution traces are segmented and abstracted into generalized, callable subroutines. As established by LRLL [143], multi-task rollout traces are abstracted via wake-sleep program clustering into parameterized Python functions, replacing ad-hoc script synthesis with reusable subroutines. CaP-X [25]

extracts modular helper primitives from successful execution rollouts, while RATs [186] organizes newly discovered exploratory controllers into a permanent, queryable skill registry.

- *AST Refactoring and Cross-Trial Controller Rewriting:* The agent iteratively modifies the underlying perception, kinematic transforms, or feedback loops of an existing controller script across trial boundaries. Act–Observe–Rewrite (AOR) [61] operationalizes this paradigm by analyzing failure tracebacks and keyframe imagery post-trial, prompting a code-generation model to refactor controller ASTs for the subsequent attempt. ASPIRE [96] extracts reusable repair snippets and execution guards from failure traces to harden brittle manipulation code.

- *Executable Policy Graphs:* Programmatic policies can make data flow and execution logic explicit through modular graphs. Graph-as-Policy (GaP) [14] synthesizes directed computation graphs with perception, planning, and control nodes, and refines candidate graphs through simulation before repeated deployment. An edge interpreter executes the resulting graph, whose perception nodes may still invoke foundation models at runtime. ENCORE [57] uses structured demonstration packages to synthesize modular robot programs, while Generalized Coding TAMP [105] leverages simulation rollouts to synthesize and freeze executable task programs evaluated across unseen task instances.

- *Multimodal Procedural Contracts:* In World Action Agent (WAA) [190], acquired skills are encapsulated as multimodal procedural packages containing explicit precondition bounds, visual anchor images of valid initial states, and verifiable postcondition checks. An offline Editor reconciles new demonstration trajectories with the pre-existing skill library, adding, pruning, or retiring skill modules based on empirical verification evidence.

The Synthesis–Execution Latency Asymmetry. Programmatic skill acquisition can decouple development synthesis from runtime execution. Compiling a policy graph or refactoring a complex controller script (as in GaP or RAPID [90]) involves model calls, program checks, and verification, with system-dependent computational and temporal costs. Once retained, the artifact can reuse synthesized execution logic, but its execution rate depends on the perception, planning, control, and model calls it invokes.
Yet, retaining an inspectable program does not entirely eliminate runtime reliance on foundation models: Astra-Sim2Real [37] retains refactored feedback loops where an agent selects continuous motion arguments online, while visual routines provide real-time stopping checks. System evaluations must rigorously distinguish deployment execution latency from the amortized computational cost of offline program synthesis.

4.3 Orchestration Harnesses, Tool APIs, and Context Builders (H_k, T_k)

Harness and tool updates modify the operational scaffolding that mediates between foundation models, sensory observation streams, and physical actors ($H_k \to H_{k+1}, T_k \to T_{k+1}$). Retained changes to prompt construction, tool schemas, or error recovery wrappers can alter later execution even when neural parameters θ and base primitives S remain fixed. They change the information available to a decision, how capabilities are invoked, or when recovery is triggered; they may also accompany updates to other components.

Harness and Tool Mutation Targets. Across the literature, harness mutations target three distinct operational mechanisms: *Context builders and prompt orchestration code* optimize the procedural functions that format sensory inputs, filter perceptual history, and construct reasoning contexts. SHAPER [152] optimizes context-building code alongside textual skill descriptors, iteratively refining which sensory features, action histories, and failure diagnostics enter the planner's prompt. AdaHVLA [138] adapts the orchestration code governing retained runtime

state, multi-camera visual context, progress commitments, and recovery decisions around a frozen VLA. *Tool interface contracts and dynamic capability routing* (T_k) alter callable API signatures, argument typing, and execution schemas. RACaP [73] executes a two-phase developmental curriculum: the first phase expands typed policy APIs and experience memory, while the second phase autonomously diagnoses task failures to mutate ReAct harness wrappers. Zetta [21] refines dynamic critic modules, recovery tools, and error-filtering routines wrapped around frozen manipulation actors, expanding functional capability without weight retraining. *Repair and execution orchestration* changes how the harness handles failures and coordinates calls. RoboFoundry [81] refines task repair routines and general harness logic using recorded pre-repair states and system-policy verifier scores. Here, verifier feedback guides harness revision; its use does not by itself establish that the verifier or acceptance gate is updated. HarnessPAI [155] and HexaAnything [27] synthesize generalized execution wrappers and multi-agent dispatch pipelines, respectively, targeting how available capabilities are coordinated.

Combinatorial Interface Regression and Upward Compatibility. Mutating H_k or T_k introduces severe risks of *combinatorial regression*. Because the execution harness serves as the universal mediation layer across all system components, modifying an observation formatter or altering an API signature can silently violate implicit invariants relied upon by pre-existing skills S_k or memory entries M_k.
As an illustrative failure mode, expanding a prompt to resolve an insertion failure could displace spatial information needed by another task when the context budget is fixed. While RACaP [73] mitigates this risk via paired candidate rollouts under identical initial seeds, and RoboFoundry [81] enforces historical trace replay, these development-time checks do not guarantee the absence of regressions. Checks used for candidate selection should be distinguished from subsequent evaluation: a selection score cannot also serve as an independent final-test endpoint. Claims of later gain should follow the evaluation boundary defined in Section 6.1.2, without imposing an additional Level 4 requirement.

4.4 Neural Parameters: Policies, Representations, and Evaluators (θ_k)

Neural weight mutations ($\theta_k \rightarrow \theta_{k+1}$) alter the continuous function approximators governing motor control, visual perception, high-level reasoning, or execution monitoring. Updating an actor changes action generation, updating a reasoner changes task interpretation or tool selection, and updating a verifier changes the signals used to assess progress and trigger intervention. The behavioral effect therefore depends on which subnetwork changes and where its outputs enter execution.

Functional Loci of Neural Adaptation. Reviewed systems target three distinct neural substrates: *Continuous motor actors and specialized action experts* update the weights governing low-level robot control chunks. RoboCat [9] fine-tunes generalist action transformers on self-generated physical rollouts, demonstrating progressive multi-task skill acquisition. RoboClaw [70] integrates automated hardware collection with automated reset mechanisms to support continuous policy training. Learning while Deploying (LWD) [157] interleaves fleet-wide autonomous rollouts with targeted human interventions, freezing the vision-language backbone while continuously updating a specialized action expert alongside value and critic networks. SOAR [203] fine-tunes goal-conditioned behavioral-cloning policies on retained autonomous rollouts while holding the high-level VLM task proposer and success detector strictly frozen. Under researcher-selected training, CapVector [132] provides a complementary parameter-transfer mechanism by merging capability vectors extracted from matched fine-tuned checkpoints into pretrained weights. SUGAR [163] instead refines human-video motion priors through physics-based learning and distills the resulting skills into a hierarchical humanoid loco-manipulation policy. *Distillation of high-level deliberation and tool routing* compresses frontier reasoning capabilities into compact edge models. Guava [86]

collects interaction traces generated by proprietary foundation models, filtering and distilling tool-use trajectories into a compact, localized VLM capable of autonomous tool dispatch without cloud connectivity. Analogously, WAA [190] and Robo-Harness K1 [74] fine-tune student vision models on historical execution logs to internalize complex tool-calling syntaxes and affordance parsing, preserving underlying analytical controllers while eliminating high-latency API dependencies. *Learned execution monitors and outcome verifiers* train neural evaluators rather than motor actors. VLA-Corrector [126] trains a neural stage and risk verifier offline using privileged simulation labels, subsequently deploying the frozen verifier to trigger runtime prompt steering around a frozen VLA actor.

Feedback Compounding and Evaluator Exploitation. Neural self-improvement architectures that couple autonomous data collection with learned success detectors face risks of *feedback compounding*. False-positive verifier outputs may admit unsuccessful rollouts into subsequent training data or distort optimization feedback. Depending on the training method, these errors can affect supervised policy fitting or reinforcement-learning updates. Optimization against an imperfect evaluator may also favor behavior that satisfies the detector without achieving physical task completion; whether this exploitation occurs requires empirical assessment. Evaluating neural self-improvement requires independent auditing of verifier calibration separate from policy success, combined with longitudinal evaluations across sequential update cycles to detect catastrophic forgetting of legacy capabilities.

4.5 Environmental Assets and Learning Procedures (E_k)

Updates to E_k modify the developmental infrastructure that guides future capability acquisition: physics simulation assets, executable reward functions, curriculum schedulers, and interactive pedagogical protocols ($E_k \rightarrow E_{k+1}$). Rather than directly producing an operational manipulation policy, mutating E_k alters the optimization landscape within which new policies are synthesized.

Environmental Mutation Paradigms. Systems targeting E_k operate across three generative mechanisms: *Autonomous evolution of executable reward code* resolves the fragility of manual reward engineering for complex manipulation. As demonstrated by Eureka [100], an agentic language model analyzes raw environment code and policy evaluation metrics to iteratively synthesize, benchmark, and mutate executable Python reward functions within massively parallelized physics simulations. By searching directly over reward code space, Eureka discovers novel shaping terms that train robust continuous policies for dexterous manipulation tasks such as pen spinning. HARBOR [72] scales this paradigm to full environment generation, synthesizing training configurations, domain randomization ranges, and reward formulations concurrently. *Generative digital twins and asset reconstruction* automate simulation construction from raw perceptual inputs. DexAgent [159] reconstructs interactive digital twins directly from monocular human demonstration videos, synthesizing metric CAD meshes, kinematic articulation joints, and physical verification bounds. HARBOR [72] similarly automates simulation asset creation, enforcing physical sanity checks on generated models prior to policy training. *Adaptive teaching and pedagogical model optimization* targets the communication protocols through which external human feedback is interpreted. Language Model Predictive Control (LMPC) [79] fine-tunes an interactive language model on historical human–robot teaching interactions, improving its responses to feedback and its generation of reward code for MuJoCo Model Predictive Control (MJPC). The trained model is a neural update target (θ_k), while its role in reward-code generation contributes to the developmental workflow (E_k), consistent with the joint N/E classification in this review.

Distinguishing Learning Resources from the Acquisition Algorithm. A fixed acquisition algorithm can still produce updates to E_k. In Eureka, for example, retaining a revised reward function changes a resource used in subsequent policy learning even if the reward-search algorithm remains

fixed. Conversely, running policy training with unchanged rewards and training procedures may update θ_k without changing E_k. For agent-guided development workflows such as ENPIRE [165], the relevant question is which resources or procedures are revised and retained. Updating a reward, curriculum, or training recipe does not by itself establish that the algorithm producing those updates also evolves.

Evaluating a retained learning procedure or simulation asset E_k requires identifying how it changes subsequent learning and manipulation outcomes. L4 requires a retained-artifact lineage and a reported manipulation gain over a frozen comparator under a source-declared independent evaluation boundary. Reductions in human effort, sample complexity, or compute are additional outcomes whose uncertainty and accounting scope should be reported; they do not replace evidence of manipulation gain.

4.6 Cross-Target Ingestion Realities and the RSI Case Analysis

The five groups of acquisition targets affect physical behavior through fundamentally different points of use (Table 3). Memory M_k supplies operational constraints or relational guidance to reasoning and planning; skills S_k provide callable procedures and compositional execution logic; harnesses and tools H_k, T_k shape visual context formatting, capability routing, and recovery thresholds; neural parameters θ_k determine learned continuous mappings for actors, representations, or verifiers; and learning resources E_k alter the optimization landscape for subsequent policy acquisition.

Diagnostic Trade-Offs: What Should Be Updated When Manipulation Fails? Because these targets operate across distinct functional layers, selecting an update target requires diagnosing the root operational cause of a physical failure:

- *Semantic and Preference Misalignment $\rightarrow$ Update M_k:* When failures arise from ambiguous task specifications, uncalibrated object spatial relations, or shifting user preferences, declarative guidance revision provides the lowest-cost remediation. Updating memory alters command formulation without incurring code compilation latency or neural fine-tuning risks.

- *Compositional and Algorithmic Breakdowns $\rightarrow$ Update S_k or H_k:* When failure stems from missing temporal branches, unhandled intermediate transitions, or brittle error-recovery logic, programmatic skill synthesis or harness restructuring provides inspectable, modular repairs. However, this assumes that the underlying perception and motion APIs possess sufficient fidelity to execute the revised logic.

- *Contact Instability and Fine-Motor Deficits $\rightarrow$ Update θ_k:* When failure is caused by frictional slip, unmodeled compliance dynamics, or force-closure grasp collapse, updating prompts (M_k) or program logic (S_k) is fundamentally ineffective. Overcoming dynamic contact failures strictly demands adapting continuous parameter weights (θ_k) through physical interaction or demonstration fine-tuning.

- *Distribution Shifts and Sparse Capabilities $\rightarrow$ Update E_k:* When the system lacks broad training data or encounters novel physical domains, modifying simulation environments, curriculum schedules, or synthetic reward code (E_k) establishes the prerequisite foundation for downstream policy learning.

As summarized in Table 3, each target presents distinct ingestion contracts and primary failure modes. In full-system deployments, a comparison of complete configuration packages supports a package-level result; attributing gains to an individual component requires a controlled ablation that isolates that specific target.

Table 3. Representative Ingestion Contracts, Points of Use, and Primary Failure Modes Across Acquisition Targets.

Target	Ingestion Contract	Point of Use	Primary Failure Mode
Memory (M_k)	Context injection or structured retrieval	Planner reasoning, constraint formulation	Epistemic drift; context saturation; stale facts
Skills (S_k)	Skill registration, program loading	Callable procedures, execution graphs	Invariant chaining collapse; kinematic mismatch
Harness (H_k, T_k)	Wrapper integration, API-schema binding	Context builders, verifier mediation	Combinatorial interface regression; token bloat
Neural (θ_k)	Checkpoint loading into inference engines	Actor, reasoner, or verifier inference	Evaluator exploitation; catastrophic forgetting
Process (E_k)	Environment synthesis, curriculum adoption	Subsequent training, sim-to-real transfer	Sim-to-real collapse; reward hacking; ungrounded dynamics

Deconstructing RSI: Local Skill Acquisition versus Complete-Task Failure. The fundamental tension between local artifact synthesis and task-level physical self-improvement is vividly demonstrated by the Robot Self-Improvement (RSI) campaign [150]. Tasked with acquiring an autonomous, long-horizon refrigerator-stocking capability through trial-and-error exploration in RoboCasa simulation, RSI achieved prolific local skill acquisition, persistently synthesizing, registering, and reusing modular manipulation primitives in its skill library (satisfying Level 3 persistence criteria). Yet, the reported refrigerator-stocking rollouts yielded zero end-to-end task successes.

The RSI campaign demonstrates why constructing, retaining, and reusing local skills does not guarantee complete-task improvement. A synthesized primitive can satisfy its own local acceptance check while leaving the physical environment in a kinematic or geometric configuration that renders subsequent operations impossible. The mechanistic lesson is that local acceptance criteria must be explicitly coupled to downstream initiation conditions, verifying whether the resulting sequence successfully executes in composition. Complete-task failure does not negate the utility of acquired local primitives; it delineates the operational boundary supported by their composition.

5. Capability Revision, Testing, and Retention

Section 4 identified the artifacts that can change. This section addresses **RQ3**: how interaction evidence guides their construction, evaluation, and retention. Expanding the update model in Equation 4, we distinguish four stages:

$$\tau_{1:N} \xrightarrow[\text{Sourcing}]{\text{Experience}} \mathcal{D}_{\text{exp}} \xrightarrow{\text{Diagnosis}} \Delta_{\text{diag}} \xrightarrow[\mathcal{U}]{\text{Synthesis}} \widetilde{C}_{k+1} \xrightarrow[\mathcal{G}]{\text{Gating}} C_{k+1}, \tag{5}$$

where traces supply an experience dataset, diagnosis proposes a change, $\mathcal{U}$ constructs a candidate, and $\mathcal{G}$ determines its adoption. This decomposition does not imply that every system implements every stage or deploys the result on hardware. Successful traces, demonstrations, or corrections may support construction without explicit failure diagnosis. Stages can also recur: an unaccepted candidate may prompt further collection, diagnosis, or revision while the current configuration remains available.

5.1 Experience Sourcing and Active Exploration

Experience sourcing determines which situations and outcomes become available for subsequent diagnosis and construction. Failure-focused collection concentrates on exposed weaknesses, while active task proposal can broaden coverage beyond repeatedly encountered situations. Physical interaction incurs execution time, wear, and safety costs, whereas simulation offers different reset and observation privileges; the value of a collection strategy therefore depends on both the information it supplies and the resources it requires.

Reactive Collection and Task Proposal. Two recurring experience-sourcing mechanisms are: First, *reactive failure ingestion* uses unsuccessful or corrected executions to initiate revision. Zero2Skill [147] and SkillWeaver [204] illustrate how feedback and exploration traces can support subsequent artifact construction. Failure-focused sampling can reveal weaknesses, but attribution benefits from successful reference executions: an observed failure alone may not identify which component should change. Second, *active task orchestration and curriculum proposal* delegates task generation directly to the agent. In fleet-scale infrastructures such as AutoRT [1], foundation models propose diverse manipulation goals across multi-robot deployments, using VLM scene descriptions and task-selection rules, including safety criteria, to filter candidate interactions. ENPIRE [165] logs trajectories, videos, and reward signals for code-revision experiments; construction of revised learning recipes is discussed in Section 5.3.
These mechanisms shape the distribution of experience supplied to acquisition: task proposal determines which goals are attempted, while exploration determines which object–skill combinations and states are encountered. Revising this selection process changes the acquisition procedure; simply executing a fixed collection schedule need not do so.

Exploration Choice and Reset Costs. RATs [186, Table 4] reports an exploration ablation in LIBERO-PRO simulation using CaP-Agent0 as the test-time execution configuration. When task exploration is directed by an intrinsic curiosity reward—favoring novel object–skill combinations at intermediate estimated difficulty—50 exploration iterations yield a post-synthesis task success rate of 32.3%, compared to 24.7% under unguided random play and 23.2% in the baseline without play. This comparison connects exploration choice to the utility of the subsequently synthesized capabilities. Its matched iteration budget does not equate physical costs or isolate which induced state differences explain the gain.
Physical experience sourcing also faces a reset constraint: a simulator can restore a specified initial state, while a robot may need additional hardware, recovery policies, or an operator to reposition objects. RoboClaw [70] develops reset behavior within its collection infrastructure. Reset behavior determines whether another useful trial can begin from the resulting physical state. Reset and rescue dependencies therefore belong to the lifecycle accounting in Section 5.5.

5.2 Failure Diagnosis and Update Attribution

An observed failure can admit several explanations, including task grounding, perception, control, memory, or verifier errors. Diagnosis connects this observation to a component that can be changed. We represent proposed targets and their supporting evidence as:

$$\begin{aligned} \delta^{(r)}_{\text{diag}} &= \left(j_r, \mathcal{E}^{(r)}_{\text{diag}}\right), \qquad j_r \in \{\theta, H, M, S, T, E\}, \\ \Delta_{\text{diag}} &= \{\delta^{(r)}_{\text{diag}}\}_{r=1}^{R}, \qquad R \geq 1. \end{aligned} \tag{6}$$

Each record pairs a proposed target j_r with diagnostic evidence $\mathcal{E}^{(r)}_{\text{diag}}$. The collection Δ_{diag} gathers R records for one candidate update; these records may address the same target or different targets. These records are an analytical representation, not a data structure required of every surveyed system. A causal hypothesis inferred from a trace is distinct from intervention evidence showing that changing the proposed cause alters the outcome.

Trajectory Summaries and Entry-Level Evidence. Diagnosis can be organized at different levels: First, *trajectory-level holistic summarization* processes multimodal rollouts. SHAPER [152] uses episode summaries to guide revision of textual skills and context code. A summary combines observations across steps, allowing the proposed revision to address dependencies between planner guidance and the context supplied to it. Second, *claim-level evidentiary ledgers* attach evidence to individual guidance entries. RoboHarn-Evo [6] distinguishes task-level and action-level knowledge and reconciles supporting, opposing, and unverified observations. Linking

observations to particular entries and their applicability conditions localizes which knowledge should be retained, withdrawn, or revised. Trajectory summaries thus support broader revision proposals, while entry-level evidence supports targeted knowledge maintenance.

Testing Diagnostic Hypotheses. A diagnosis can target the wrong component: changing a perception threshold, for example, may leave a contact-control failure unresolved. This is a design risk rather than an inevitable consequence of a particular diagnostic representation.
AHA [23] uses FailGen to perturb successful simulated trajectories and produce labeled failure examples for training a failure-reasoning model. Controlled perturbations support those training labels; a model's explanation of a new failure does not itself verify a causal repair. Lifecycle reports should distinguish generated failure labels, inferred hypotheses, and interventions that test whether a proposed repair restores task progression.

5.3 Candidate Construction

Candidate construction converts experience and proposed diagnoses into a concrete artifact. It may aggregate the records in Δ_{diag} to revise one component or several together, or construct candidates directly from demonstrations and successful traces. The central mechanism is the transformation from feedback or training data into revised guidance, code, parameters, or learning resources.

Update Operators. The target representation determines the available update operations; the following examples illustrate representative families rather than an exhaustive list. As in Section 4.3, tool-contract revisions (T_k) are discussed alongside harness changes (H_k), while remaining distinct targets in C_k. Edits to callable signatures, schemas, or API bindings construct $\widetilde{T}_{k+1}$ candidates.
First, *two-stage textual-guidance and harness optimization* ($\mathcal{U} \to \widetilde{M}_{k+1}, \widetilde{H}_{k+1}$) separates revision of planner guidance from revision of context-building code. SHAPER [152] calls the former "skill evolution": it first revises textual procedural guidance with the seed harness fixed, then revises context code with the selected guidance fixed. Under our artifact taxonomy, these updates target M_k and H_k, respectively. The staged procedure uses a fixed counterpart at each step, first changing the guidance available to the planner and then changing how execution information is presented to it. Second, *knowledge-entry revision and ledger reconciliation* ($\mathcal{U} \to \widetilde{M}_{k+1}$) revises stored conditions and strategies rather than executable code. RoboHarn-Evo [6] constructs Task and Action Knowledge, merges equivalent entries, associates evidence with content versions, and withdraws challenged versions from retrieval. Revised strategies remain candidates until applicable executions support them without opposing evidence. Construction and evidence maintenance are coupled: a revision changes the stored strategy, while its version-specific evidence determines whether it becomes available for retrieval. Third, *data aggregation and policy-checkpoint training* ($\mathcal{U} \to \widetilde{\theta}_{k+1}$) updates parametric actors through gradient-based training. Systems such as RoboClaw [70] pair forward task executions with autonomous physical resets, synthesizing candidate policy checkpoints $\widetilde{\theta}_{k+1}$ using a conditional flow-matching objective on collected action trajectories. Here, collected trajectories influence later actions through fitted policy parameters rather than through a retrieved instruction or an explicit program edit.
Fourth, *revision of developmental assets and learning procedures* ($\mathcal{U} \to \widetilde{E}_{k+1}$) constructs proposed reward code, simulation configurations, curricula, or training implementations. Eureka [100] uses environment code and policy-training statistics to generate and refine reward programs. ENPIRE [165] uses rollout trajectories, videos, and reward feedback to modify training implementations and recipes, sharing successful revisions across parallel agent–robot experiments. These revisions produce $\widetilde{E}_{k+1}$ candidates; collecting data or fitting policy weights under a fixed recipe does not revise that recipe. Policy checkpoints trained using this developmental candidate are separate neural artifacts (θ).

Synthesis Interfaces and Validation. Executable skill synthesis ($\mathcal{U} \rightarrow \widetilde{S}_{k+1}$) generates or revises program implementations around domain-specific interfaces. CaP-X [25], ASPIRE [96], and GaP [14] use supplied robot or simulator APIs for these constructions. The supplied interfaces define the operations a candidate can call and the arguments it must provide. Syntax checks assess program form, simulation execution tests behavior under modeled conditions, and physical validation examines behavior on the robot; none substitutes for the others. Interface-constrained synthesis therefore does not by itself certify all generated programs or guarantee physical safety. Across these update operators, construction produces a candidate whose adoption is considered next. Component comparisons are needed to distinguish the contribution of one target from the effect of a combined update.

5.4 Promotion, Regression Checks, and Rollback

The promotion rule in Equation 4 may combine admissibility checks with a candidate–current comparison on validation data. Its metric, threshold, and any risk penalties should be explicit. As specified in Section 2.3, validation can guide refinement; it is distinct from a protected final test.

Complementary Validation Safeguards. Three complementary safeguards organize the gating mechanisms and deployment requirements. Their presence and empirical coverage must be established separately for each system: First, *static contract and sandbox admissibility* is a proposed pre-execution safeguard. Syntax and API checks, restricted I/O, resource limits, and timeouts can reject inadmissible candidates within a specified interface. SHAPER [152] reports a required context-function interface, restrictions on imports, file I/O, and dynamic execution, and a runtime timeout before rollout. A timeout does not prove termination of arbitrary generated programs, and a configured memory limit does not establish a program-level bound. RoboFoundry [81] reports trace-based edit evaluation and promotion, without establishing these static or sandbox guarantees. Second, *historical trace replay and regression assessment* examines broader effects of a candidate. In RoboFoundry [81], recorded pre-repair states from other tasks are replayed to propose task-level edits; a system-policy verifier compares their capability-gap scores with recorded successful repairs, and a general edit is promoted when the average score difference is nonnegative. This is a trace-based proxy, distinct from rerunning prior manipulation tasks to measure regression. RACaP [73] instead uses paired parent–candidate evaluations under matched task and seed conditions to assess the effect of a retained update. Third, *execution-grounded evidence* checks whether a proposed update is supported by relevant interaction outcomes. In RoboHarn-Evo [6], a knowledge entry becomes eligible for retrieval when applicable executions provide support without opposing evidence; a challenged entry is withdrawn, and an untested revision remains a candidate. This internal admission rule is distinct from final independent evaluation. The source reports repeated acquisition with frozen held-out tests in simulation, alongside a separate real-robot knowledge-update experiment.

Rejected Candidates and Rollback. Candidate rejection preserves the current configuration before adoption; rollback restores an earlier version after an update has been adopted. The latter requires keeping the relevant artifacts and their compatible dependencies available. Restoring software or model parameters does not restore the physical scene: a displaced object, occupied gripper, or changed contact state may require separate recovery before execution can resume.
Incomplete reporting of rejected candidates and rollback events limits assessment of an update process. Published checkpoint curves, including collection-and-training results in RoboClaw [70], describe the evaluated trajectory; they do not by themselves reveal all unsuccessful candidates or the cost of selecting the reported checkpoints. Missing rejection logs cannot establish concealed human debugging or prove that the underlying task distribution was artificially restricted.
Physical deployment requires *fail-safe rollback protocols* as a design objective. AgenticRobotics [180] reports evidence-gated promotion and transaction recovery in LIBERO-10

simulation, and identifies independent safety interlocks as necessary for real-robot deployment. Its study does not demonstrate rollback following joint vibration or physical safety stops. Workflow recovery therefore supplies evidence about software coordination, while controller safety and physical recovery require separate hardware evaluation.

Deployment and Return to Development. Adoption connects the retained artifact to a point of use. Runtime artifacts enter execution through retrieval, skill invocation, orchestration, or model inference (Section 3). Learning resources E_k instead enter subsequent acquisition through the retained asset or procedure. Their effect on manipulation is mediated by the resulting capability, such as a trained policy checkpoint, and its later execution (Section 4.5).
When hardware wear, sensor drift, or novel objects produce persistent deployment failures, a system may return to development. Evaluating repeated self-improvement requires tracing this transition and its interventions. Engineer-led recalibration or reward revision can support capability acquisition, but its contribution and cost must be included when judging the autonomy of the complete cycle.

5.5 Supervision and Resource Budgets

Acquisition mechanisms draw on different forms of supervision and resources. Automated candidate generation may still depend on supplied demonstrations, operator resets, or engineer-led repairs elsewhere in the lifecycle. These dependencies affect which mechanisms can be repeated under a given deployment budget. Table 5 compares protocol outcomes and dependencies; the supplement preserves their detailed lineages.

Budget Components. We organize resource accounting along four dimensions:

$$\mathbf{B}_k = (B_{\text{human}}, B_{\text{oracle}}, B_{\text{wear}}, B_{\text{compute}}).$$

These dimensions form an accounting profile rather than a sum in common units. Relevant quantities depend on the acquisition mechanism; privileged access may require a description of supervision and permissions rather than a scalar cost. First, *Human mechanical and communicative labor* (B_{human}) encompasses all external interventions across the learning loop, including initial teleoperation demonstrations, mid-rollout policy rescues, manual workspace resets, and out-of-band code refactoring. Operator-dependent sample efficiency should be reported together with its reset budget when judging autonomy. Second, *Privileged sensory and simulation access* (B_{oracle}) accounts for the provenance of ground-truth supervision; simulation-based development [72, 100] can provide state variables, reward signals, and reset operations that are unavailable under the corresponding robot interface. The relevant distinction is which privileges are available during acquisition and which remain available during later execution. Third, *Physical wear and hardware interaction budgets* (B_{wear}) describe interaction exposure and hardware demands. Depending on the platform and available instrumentation, relevant records may include contact time, joint torque saturation events, or emergency safety trips; these are indicators of exposure and operating conditions rather than interchangeable measures of wear. Finally, *Computational and token expenditure* (B_{compute}) describes resources consumed across candidate construction, reflection, training, and selection. Suitable measures may include model calls or API tokens, training compute, and elapsed time, with the accounting scope stated for the reported process.

Lifecycle Synthesis. Experience selection, diagnostic evidence, candidate construction, and promotion determine different parts of the acquisition process. RATs changes the exploration supplying later synthesis; SHAPER separates guidance and context-code revision; RoboHarn-Evo couples knowledge maintenance to entry-level evidence; and RoboFoundry screens edits with trace-based scores. The same update target can therefore be reached through different acquisition

processes, and similar retained artifacts need not imply similar costs, reliability, or conditions of reuse. Connecting how an artifact was obtained to how it is later used supports the comparison of outcomes and evidence gaps in Section 6.

6. Empirical Realities: Capability Lineage and the Evidence Ledger

This section addresses **RQ4**: which reported experiments support retained capability gains, and what further evidence is needed for repeatable acquisition? Across the 162-record corpus, we distinguish execution robustness from persistent artifact utility and procedural repeatability. Each credited chain links an artifact to its acquisition process, later use, comparator, and reported evaluation boundary. The strength of that boundary and the scope of the resulting claim are assessed separately.

6.1 Evaluation Designs and Attribution Boundaries

We distinguish three evaluation questions: execution performance under a given configuration, the utility of retained artifacts in later use, and the repeatability of the acquisition procedure (Figure 5).

6.1.1 Auditing Agent-Mediated Execution (Track A)

An execution evaluation assesses whether an embodied agent completes goal-directed physical tasks through its active decision and control interface under a given configuration C_k. Metrics including nominal task success rate, human intervention frequency, completion latency, and execution energy must be audited alongside physical reset dependencies, retry limits, and available tool repertoires.

A successful runtime recovery demonstrates a response to the current task, but does not by itself establish a retained capability. If the correction is preserved for subsequent use, that additional process belongs to acquisition analysis. RoboMemArena [64] evaluates memory-dependent long-horizon tasks in simulation and the real world, where within-episode memory use remains distinct from cross-episode retention. Comparisons intended to isolate the execution architecture should control the actor, observations, initial conditions, and retry budgets as appropriate to the claim. Broader system comparisons can establish package-level effects without attributing the gain to one execution component.

6.1.2 Auditing Retained Capability Gains (Track B)

An acquisition evaluation identifies which constituent of $C_k = (\theta_k, H_k, M_k, S_k, T_k, E_k)$ changes, how the resulting artifact is retained and used, and whether later manipulation improves over a frozen comparator under declared test conditions. A comparison of complete configuration packages supports a package-level result; attribution to an individual component requires a comparison that isolates that component.

To qualify for **Level 4 (Measured Manipulation Gain)**, one experimental chain must establish artifact construction, persistence, later use, a reported manipulation gain, and a source-declared separation between acquisition or selection and the credited evaluation. Held-out instances, acquisition/test seed partitions, fixed cross-domain transfer, or explicitly separated stages with declared evaluation conditions can establish this minimum boundary. Mere chronology—collecting data, training, and then reporting a score—does not suffice. A score used to construct or select the credited artifact cannot also serve as its independent final-test endpoint.

This source-report classification is distinct from the stricter requirement that every final-test channel be explicitly excluded from construction and selection. The boundary audit reports both criteria separately. Statistical significance is not a universal L4 threshold: sample size,

uncertainty, experimental scope, and component attribution qualify the strength of each reported gain. Incomplete documentation remains unresolved evidence rather than proof of leakage.

6.1.3 Auditing Acquisition as a Repeatable Procedure

Evaluating repeated execution rollouts measures the runtime variance of a single static artifact; evaluating independent acquisition histories measures whether the underlying self-improvement mechanism $\mathcal{U}$ reliably produces useful capabilities from declared initial resources. These may include shared pretrained models, source archives, or skill libraries; the restarted and retained components must be identified.

Procedural comparisons should identify the initial resources, independent acquisition histories, and rules for selecting or rejecting updates. Acquisition repeatability and human dependence are separate properties: repeated runs can still rely on demonstrations, resets, or engineering support. Their joint assessment uses the intervention criteria and accounting periods defined in Section 2.4.

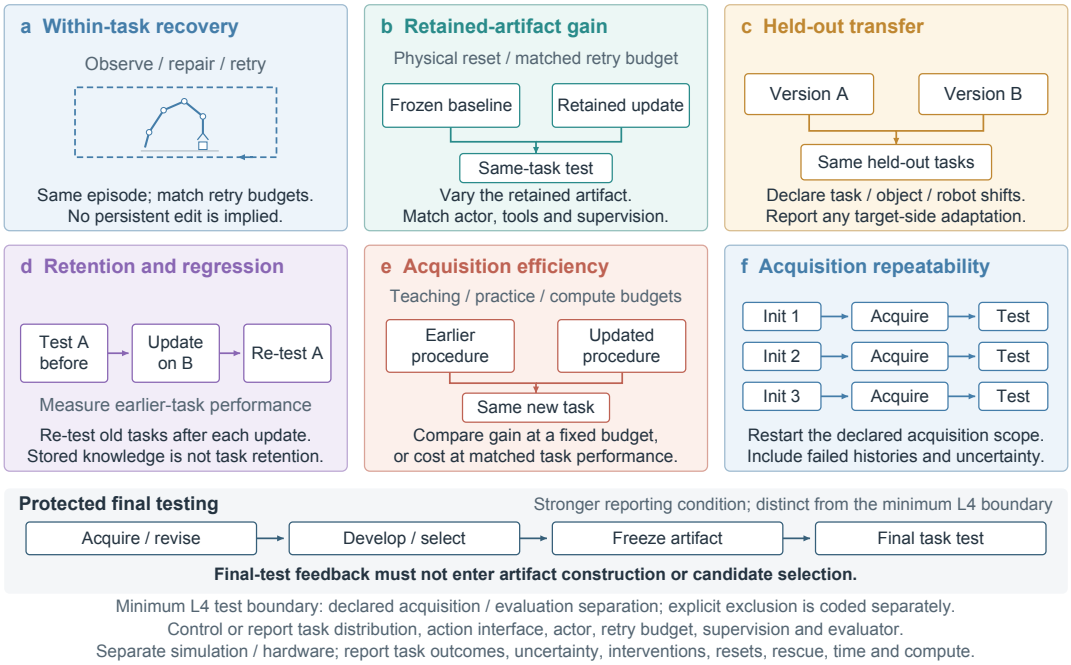


Figure 5. Taxonomy of experimental evaluation protocols and attribution boundaries. Panels (a)–(e) formalize distinct empirical endpoints: within-episode reactive recovery, retained-artifact deployment gain over a frozen baseline, out-of-distribution task transfer, sequential retention and backward regression, and resource efficiency. Panel (f) evaluates procedural repeatability by restarting the candidate generation mechanism across independent operational histories. The lower boundary highlights the strict evaluation redline: complete isolation of the evaluation partition $\mathcal{D}_{\text{test}}$ from both candidate synthesis $\mathcal{U}$ and promotion gating $\mathcal{G}$, alongside quantified accounting of human interventions and physical resets.

6.2 Evidence Levels in the Reviewed Corpus

Applying our cumulative evidentiary criteria across the 162 primary studies assigns 9 records to Level 1 (Execution Baseline), 18 to Level 2 (Capability Construction), 70 to Level 3 (Persistence and Later Use), and 65 to Level 4 (Measured Manipulation Gain). Crucially, zero inspected experimental chains satisfy the joint Level 5 criterion under this review's intervention-accounting requirement (Table 4).

These mutual distributions reveal three profound empirical realities across current self-improvement research: First, *the Level 3 bottleneck (70 records):* Nearly half of the surveyed

Table 4. Five-Level Evidence Framework for Robotic Self-Improvement. Levels describe supported claim endpoints; L5 adds a review-defined joint criterion rather than a universal system ranking.

Level	Evidence Endpoint	Mandatory Evidentiary Requirement	Exclusionary Criterion (Insufficient Alone)	Corpus Count
L1	Execution Baseline	Agent-mediated manipulation without established construction of a capability artifact.	Within-episode reactive recovery, prompt retries, or transient state updates.	9
L2	Capability Construction	A capability artifact is constructed or adapted; persistence and later use are not both established.	Code compilation, AST generation, or mock simulation without deployed execution.	18
L3	Persistence and Later Use	An artifact is retained beyond its originating episode or acquisition stage and used in later execution or learning. Learning uses connect to the resulting capability and its later execution.	Ephemeral cache, un-ingested database expansion, or state cleared upon system reset.	70
L4	Measured Manipulation Gain	A retained artifact yields subsequent manipulation gain over a frozen comparator under a source-declared separation of acquisition or selection from evaluation.	Unchecked library accumulation, search peak on development seeds ($\mathcal{D}_{\text{val}}$), or evaluations lacking baseline isolation.	65
L5	Repeatable Self-Improvement	Independent acquisition histories yield protected-test gains, transfer, earlier-task performance assessed for retention and regression, and documented low human dependence under stated intervention criteria and accounting periods.	Single acquisition history, training-task evaluations lacking cross-task transfer, unverified earlier-task retention, or undocumented manual resets.	0
Total Audited Studies Across Corpus: 162				

literature successfully synthesizes persistent computational artifacts and demonstrates their later invocation, yet stops short of Level 4. This is predominantly caused by evaluation methodology rather than system failure: many studies evaluate self-evolving frameworks without an explicit frozen-comparator baseline, report adaptive multi-turn evaluations that obscure causal attribution, or mutate multiple components simultaneously. L3 status confirms that capability persistence is technically established, while highlighting the absence of isolated causal attribution.

Second, *the simulation-to-hardware disparity in Level 4 (65 records):* Among the 65 records establishing measured capability gains under declared evaluation boundaries, 36 have credited simulation endpoints, 22 report hardware endpoints, and 7 demonstrate gains in both environments. As analyzed below, hardware gains are heavily concentrated in modular skill additions or prompt tuning around frozen foundation models, whereas neural parameter updates dominate simulation benchmarks.

Third, *the Level 5 hardware frontier (0 records):* The absence of Level 5 hardware credentials is not an indictment of existing algorithms, but an exposure of physical reset constraints. While repeated acquisition, out-of-distribution transfer, and earlier-task retention occur jointly within bounded simulation streams (such as Conquer [148]), sustaining an autonomous self-improvement loop on physical hardware inevitably encounters manual human resets, safety interventions, and physical workcell wear. Because literature routinely omits quantified human labor accounting (B_{human}), true autonomous self-improvement on real-world robots remains an unestablished scientific frontier.

6.3 Diagnostic Case Analysis and Attribution Boundaries

To interrogate attribution boundaries, experience dependencies, and adverse failure modes across diverse execution interfaces, we examine eleven diagnostic protocols (Table 5). Rather than representing an exhaustive catalog, these diagnostic cases provide critical contrasts across acquired artifacts, candidate generation mechanisms, and evaluation protections.

The boundary audit covers all 65 L4 records and 58 L3 records with a reported-gain field, for 123 records in total. Sixty-five meet the minimum declared-separation rule used for L4, while 17 explicitly document the stricter exclusion of the final test from both construction and selection. This difference is a reporting sensitivity, not evidence that the remaining gains are absent or necessarily affected by leakage. L3 gain-bearing records lack a complete credited artifact-to-evaluation chain because of unresolved boundaries, adaptive evaluation, or package-level attribution limits.

For instance, KnowBody [95] establishes a robust physical hardware gain (12/16 versus 4/16 successes) by deploying an offline-curated package of body models and harness wrappers with cross-episode updates disabled; this confirms the utility of the static package, but does not isolate the independent causal contribution of body memory. Similarly, SOAR [203] demonstrates substantial empirical improvements (28% to 57% success), but its reported experimental lineage presents a continuous collection–training–evaluation sequence lacking an explicit, pre-isolated evaluation boundary. CaP-X [25] demonstrates notable physical hardware transfer, yet its simulation interface distinctions do not satisfy a formal held-out instance partition.

6.4 Cross-Mechanism Empirical Synthesis

Retained Knowledge and Programmatic Skills (M_k, S_k). When underlying continuous motor actors remain strictly frozen, mutating declarative memory or programmatic skill libraries regularly yields substantial manipulation improvements. RATs [186] provides a canonical plug-in contrast: injecting an autonomously discovered skill library into a frozen CaP-Agent0 baseline raises LIBERO-PRO simulation success from 23.2% to 32.3%, with physical hardware performance increasing from 30.0% to 38.8%. However, because the complete RATs pipeline concurrently refines harness routing, the holistic gain cannot be causally attributed to the skill library alone.

Recursive Harness Distillation (RHD) [60] isolates the impact of retained textual guidance: deploying its refined operational playbook to an identical lightweight agent elevates success on 48 held-out instances from 43.8% to 66.7%, whereas deploying an unrefined initial playbook collapses performance to 31.3%. This stark contrast demonstrates the extreme sensitivity of frozen actors to prompt quality, yet leaves the causal contribution of individual heuristic rules un-isolated.

SHAPER [152] provides an exemplary component-level dissection across update targets. Its textual skills represent procedural planner guidance (M_k), while its context-building code belongs to the harness (H_k). On an isolated 800-episode VLABench test partition, the baseline seed, skill-only evolution, harness-only evolution, and joint skill-harness evolution achieve success rates of 28.25%, 33.50%, 30.50%, and 34.50%, respectively. Crucially, textual-guidance revision yields the primary performance delta (+5.25%), while compounding the evolved harness confers an incremental +1.00 percentage point gain. These findings substantiate our first major insight: *in agentic manipulation, optimizing high-level command formulation and discrete skill selection frequently produces the dominant share of task gains, even while low-level motor execution remains completely static.*

Neural Adaptation and Supervision Lineages (θ_k). While programmatic systems update symbolic syntax, neural self-improvement updates continuous tensor representations. RoboCat [9] isolates experience mechanisms by holding model capacity and an initial 500-demonstration budget fixed: incorporating autonomously generated rollouts yields consistent performance gains across four physical hardware task variants. Its Level 4 hardware credential derives from an isolated fine-tuning chain, where real-world success rises from 0% to 82% and 88% with 100 and 500

Table 5. Audit of Eleven Diagnostic Protocols Across Targets and Evaluation Boundaries. The matrix summarizes retained artifacts, baseline comparators, verified endpoints, and key methodological limitations.

Protocol & Target	Retained Artifact	Comparison & Measured Gain	Test Boundary & Critical Limitation
ASPIRE [96] (S_k)	Debugged Python skills	Empty vs. learned library; gains on long tasks (sim)	Disjoint debug/test seeds; single acquisition history.
CLS [62] (S_k)	Control program primitives	*Best4* archive vs. cold-start: 57.0% vs. 45.2% (sim)	Train/val/test split; requires 15 demos; shared archive.
ENCORE [57] (S_k)	Modular robot programs	K_0 vs. K_3: 2846/3000 vs. 2889/3000 (sim); real transfer	Sealed test states; local collapse under perturbation ($50/50 \rightarrow 0/50$).
RACaP [73] (H_k, T_k)	Policy APIs & ReAct wrappers	Phase-1 vs. Phase-2: $32.8\% \rightarrow 45.0\%$ over 180 episodes (sim)	Target feedback excluded; single search run; manual curriculum.
RHD [60] (M_k)	Distilled text playbooks	Baseline vs. playbook: $43.8\% \rightarrow 66.7\%$ on held-out tasks (sim)	Partitioned environments; initial playbook harmful (31.3%).
RoboHarn-Evo [6] (M_k)	Action-effect knowledge	Empty vs. acquired store: $48.3\% \rightarrow 75.0\%$ (sim); $41.7\% \rightarrow 62.5\%$ (real)	3 sim histories; read-only test; legacy-task retention untested.
SHAPER [152] (M_k, H_k)	Textual guidance & context code	Seed vs. joint package: $28.25\% \rightarrow 34.50\%$ (800 sim episodes)	Disjoint test split; candidate branches share lineage.
WAA [190] (S_k, θ_k)	Multimodal skill contracts	Static vs. evolved skills: $43.3\% \rightarrow 75.6\%$ mean success (sim)	Target feedback excluded; single acquisition run; requires demos.
Zetta [21] (H_k)	Critic & recovery routines	Nominal actor vs. evolved harness: task gains on held-out tasks (sim)	Disjoint test seeds; physical reset and retention unmeasured.
RSI [150] (S_k, H_k)	Local manipulation primitives	Paired local grasp gains; subsequent rollouts yield **0% success** (sim)	Local unit acceptance; 113 manual harness repairs.
SkillWeaver [204] (M_k, θ_k)	Experience memory & data	Memory accelerates search; robust to multiple perturbations (sim)	Test isolation unresolved; language-plus-object perturbation: 97.8% vs. 89.4%.

demonstrations, respectively, under rigorous checkpoint-selection boundaries. However, because earlier evaluation trajectories are cyclically recycled into subsequent training pools, evaluation isolation remains phase-dependent rather than structurally permanent. EmbodiRSI [63] verifies the benefits of adaptive data collection over static rollouts under matched simulation budgets prior to human-in-the-loop refinement.

VLA-Corrector [126] updates neural verifiers rather than the primary actor, training an offline stage/risk verifier to trigger runtime prompt steering around a frozen VLA, achieving an 88.3% versus 82.5% hardware success rate. This comparison locates the acquired change in an offline-trained verifier whose outputs guide later execution around the frozen actor.

Robo-COP [189] explicitly tests when neural weight adaptation confers benefits beyond memory adaptation. Post-deployment evaluation across held-out physical initializations demonstrates mean success of 50.0% versus 38.3% for the memory-only baseline, confirming the distinct value of parameter adaptation. Its evaluation also supports candidate rejection as a safeguard against harmful updates in the tested setting.

EmbodiedRSI [127] illustrates how target utility interacts with task structure: in component ablations holding actor capacity and interaction budgets fixed, the relative ranking of code-only and skill-only adaptation completely reverses between atomic manipulation tasks and unseen composite tasks. Furthermore, its simulation-evolved harness elevates physical hardware success from 46.0% to 71.3% with a frozen SmolVLA recipient. However, because the base recipient required 50 human demonstrations per task, zero-shot harness transfer must not be conflated with zero human engineering investment.

Continual Retention and Parameter Interference. Memory Anchors [22] selects replay experiences using latent-state overlap and disagreement between old and new actions. Its reported 63% reduction in forgetting concerns selected high-conflict task pairs. Negative backward transfer measures earlier-task performance loss, so a smaller value indicates less forgetting. These findings support conflict-aware replay under the evaluated tasks and budgets, rather than a claim that memory expansion alone guarantees retention.
COLADA [30] limits direct overwriting of existing adapters through modular parameter isolation, learning task-specific ACT-LoRA adapters from dialogue-guided demonstrations while freezing the shared base model. On real-world sandwich assembly, it completes 13/16 multi-step trials, with legacy skills succeeding in 44/48 trials, compared with 47/48 during the interaction phase. These results document reuse of earlier skills under modular parameter isolation, while the reported 44/48 and 47/48 outcomes should remain visible when assessing retention. Human teaching remains part of the acquisition process.

Autonomous Data Collection versus Physical Reset Realities. AquaBot [87] optimizes speed parameters over demonstration-trained policies, reducing real-world sorting times from 37.47 to 31.34 seconds. In contrast, VaPRL [122] and MEDAL++ [123] synthesize autonomous reset policies that restore physical workcells between manipulation trials. Reset mechanisms can reduce externally supplied assistance between trials. Their net lifecycle savings depend on setup, teaching, maintenance, and residual interventions; the mechanism alone does not establish whether total human effort increases or decreases.

6.5 Repeated Acquisition, Transfer, and Retention

To separate demonstrated components from their joint support, we compare six diagnostic protocols across acquisition repetition, final-test separation, transfer, retention, and intervention reporting (Table 6).
Conquer [148] combines repeated acquisition, protected evaluation, future-task transfer, and earlier-task retention in a simulated object-pushing stream. Three independent training runs traverse 14 tasks, with stage-wise evaluations excluded from training and library updates. Final success is $95.6 \pm 0.6\%$, compared with $89.2 \pm 0.6\%$ for the Scratch comparison; reported forward transfer is 11.3%, and backward transfer is approximately zero. The source specifies an automated retrieval–training–update procedure and interaction budgets. This supports repeatable simulated self-improvement when procedural automation and stated interaction costs suffice. L5 remains unresolved under this review's additional requirement for a measured in-run intervention criterion and accounting period. This is a documentation boundary, not evidence of high human dependence. Separate hardware demonstrations do not inherit the simulation acquisition histories.
RoboHarn-Evo [6] evaluates three independent simulation acquisition histories starting from empty knowledge stores ($M_0 = \emptyset$), with 80 acquisition rollouts per history and read-only evaluation on 20 held-out configurations. Success rises from 48.3% to 75.0% for one model and from 70.0% to 88.3% for another. Frozen RMBench knowledge also transfers to RoboDojo tasks in simulation. Its separate hardware comparison reports 41.7% versus 62.5% success, but does not inherit the simulation experiment's three acquisition histories. Knowledge-entry retention does not substitute for re-evaluating earlier manipulation tasks after successive updates.
Continual Language-to-Skill (CLS) [62] executes three independent target-skill acquisition histories conditioned on a pre-existing source archive, reporting $57.0 \pm 7.8\%$ target success compared to $45.2 \pm 10.3\%$ without source conditioning. Generalized Coding TAMP [105] reports five independent program synthesis histories per environment evaluated on hidden states, while LRLL [143] executes three complete developmental curricula under varying teacher seeds, achieving 93.5% backward-transfer retention.
These protocols provide evidence for repeated acquisition under different initial resources and restart scopes. Some retain a pre-existing source archive, while others restart training or program

Table 6. Component Evidence and Methodological Boundaries in Candidate Level-5 Protocols. Six representative systems are audited across the five constitutive requirements of repeatable physical self-improvement. (S: Simulation; H: Physical Hardware).

Protocol	Repeated Acquisition	Isolated Final Test	Forward Transfer	Legacy-Task Retention	Critical Boundary / Cost Deficit
Conquer [148]	3 independent 14-task training runs (S)	Stage tests excluded from library updates	Future-task transfer over frozen baseline (S)	Near-zero backward transfer; re-tested (S)	In-run intervention criterion and accounting period unresolved; S repetitions do not establish H repetitions.
Robo Harn-Evo [6]	3 empty-store histories ($M_0 = \emptyset$); 80 rollouts (S)	Read-only snapshots on 20 held-out scenes	Zero-shot transfer to RoboDojo tasks (S)	Knowledge entry retention; task re-test unresolved	Independent H acquisition histories and total human setup labor unresolved.
CLS [62]	3 target acquisitions sharing 1 source archive	Frozen code; test cases strictly excluded	Archive-assisted gain on 9 target tasks (S)	Code preserved; target-to-source re-test unresolved	Requires 15 human demonstrations; initial setup costs unmeasured.
LRLL [143]	3 four-cycle experiments with teacher seeds (S)	Cycle-end evaluations on unseen tasks	Compositional transfer across task streams (S)	Old-task success verified: 93.5% vs. 78.9%	Relies on scripted teacher and hints; execution labor unquantified.
Memory Anchors [22]	3 simulation task orders; H unrepeated	Fixed final epoch on randomized OpenJar (H)	Sequential transfer across task pairs	Negative backward transfer measured; lower values indicate less forgetting	Human demonstrations supplied; full reset budget unresolved.
COLADA [30]	16 participant teaching runs; repeats unresolved	Multi-day held-out evaluation on real robots	Same-sandwich recipe; unseen-task unresolved	Old skills preserved across 44/48 trials (H)	Human-in-the-loop teaching; deployment help differs from acquisition.

synthesis within supplied environments. Their positive simulation results do not jointly establish independently repeated hardware acquisition, transfer, earlier-task retention, and low human dependence under a declared intervention criterion.

6.6 Observed Losses and Unresolved Evaluation Boundaries

Aggregate gains can coexist with condition-specific losses or failed complete tasks (Table 7). These comparisons identify different limitations and should not be pooled into a single estimate of regression or autonomy.

Condition-Specific Losses and Update Trade-offs. ENCORE [57] reports an aggregate simulation gain alongside a plate-condition decrease from 50/50 to 0/50 under a tested perturbation. SkillWeaver [204] reports 89.4% success for its augmented-data policy versus 97.8% for the original-data policy under the language-plus-object perturbation, alongside benefits under other perturbations. This is a condition-specific training-mixture comparison, not evidence of a general nominal-performance decline or sequential forgetting.
Policy-Level RSI [36] repairs 19 baseline failures while introducing eight regressions in its reported RoboTwin comparison. Such results motivate testing both repaired and previously successful cases. They do not establish that all specialization causes forgetting or identify one universal cause of update-induced loss.

Local Acceptance and Complete-Task Outcomes. The RSI campaign [150] reports a locally accepted approach that satisfies a spherical reachability check but leaves the subsequent grasp

Table 7. Observed Losses Alongside Acquisition Benefits. Each row describes a reported comparison and its interpretation boundary.

Protocol	Reported Adverse Comparison	Supported Interpretation	Causal Inference Boundary
ENCORE [57]	Plate condition under physical perturbation: K_0 (50/50) $\rightarrow$ K_3 (0/50) success.	**Catastrophic perturbation collapse**: Retained program update fails under environment distribution shift despite aggregate gains.	Condition-specific failure; does not quantify general degradation rate across unperturbed tasks.
RATs [186]	Bimanual handover task: 12/50 before vs. 10/50 after skill acquisition.	**Task-level variance discrepancy**: Local primitive performance deviates downward from positive aggregate benchmark trends.	Small count difference; statistical significance is not established here.
RHD [60]	Initial playbook degrades success to 31.3%; refined playbook improves it to 66.7%.	Initial and refined guidance have different effects under the same recipient; individual harmful rules are not isolated.	Artifact version disparity; reflects candidate initialization sensitivity rather than sequential forgetting.
SkillWeaver [204]	Original vs. augmented policy: language-plus-object perturbation drops from 97.8% to 89.4%.	A condition-specific loss alongside gains under other perturbations; this is not a nominal-distribution comparison.	This training-mixture comparison does not establish sequential old-task regression.
RSI [150]	Validated local grasp improvements followed by 0/60 successes on whole-task stocking rollouts.	A successful local repair is insufficient for complete-task success.	Local and whole-task outcomes measure different capabilities; no decline from a successful whole-task baseline is established.

unable to reach. This identifies a mismatch between an acceptance criterion and the next skill's requirements. Separately, a later local grasp improvement is followed by 0/60 whole-task successes. The latter result establishes that a local repair is insufficient for complete-task success; it does not establish a decline from a previously successful whole-task baseline or isolate the cause of every failed rollout.

Documented Human Dependence and Missing Cost Measurements. RSI explicitly reports 113 human harness repairs [150]. These document an external engineering dependency rather than concealed assistance, and should be distinguished from physical hardware resets. Real2Gym's reported execution savings do not quantify the complete cost of constructing the simulator and extracting skills [118]. Such accounting boundaries prevent an end-to-end cost conclusion; missing measurements remain unresolved rather than being treated as zero or as evidence of hidden labor. These findings motivate separate measurements of task transfer, update-induced loss, sequential old-task retention, complete-task success, and acquisition costs. Each interpretation should remain tied to the specific comparison and experimental setting that supports it.

Table 8. Comparative Overview of the 162 Reviewed Studies.

G.	Study	Agent interface	Capability artifact	Setting	Acquisition	Level
A. Agent-Mediated Manipulation (53 records)						
A	2AM [41]	Skills	N: action model	Sim	Offline training	L3
A	ABot-Claw [49]	Mixed	M: scene/trace memory	HW	Interaction	L3
A	Agentic Robot [177]	Skills	N: verifier LoRA	Sim	Offline training	L3
A	AGP [54]	Code	S/M: code/experience	HW	Demo + feedback	L4(H)
A	CAM [202]	Skills	N/S: verifier/monitor	Sim+HW	Train + synthesize	L3
A	CaP-X [25]	Code	S/N: utilities/policy	Sim+HW	Synthesis + RL	L4(H)
A	Code as Policies [78]	Code	S: policy functions	Sim+HW	Task-time synthesis	L2
A	CoPAL [56]	Skills	State: episode plans	Sim+HW	Task-time planning	L1
A	Cortex [113]	Skills	N: planner/actor	Sim+HW	Offline training	L3
A	DoReMi [33]	Skills	N: detector LoRA	Sim+HW	Offline training	L4(B)
A	DRAE [94]	Integrated	M+N: experts	Sim+HW	Routing + RL	L3
A	Goal2Skill [93]	Skills	N: skill policies	NR	Offline training	L3
A	GPT Policy [17]	Commands	M: supplied demos	HW	External demos	L3
A	HarnessVLA [191]	Mixed	M/S: memory/cards	Sim+HW	Explore + compile	L4(S)
A	HERO [69]	Skills	M+N: skill library	HW	Orchestrated RL	L3
A	Hi Robot [124]	Skills	N: hierarchy weights	HW	Labels + training	L4(H)
A	Inner Monologue [46]	Skills	N: success predictor	Sim+HW	Offline training	L3
A	K1 [74]	Commands	N: command LoRA	Sim	Teacher distillation	L4(S)
A	KnowNo [117]	Skills	H: decision threshold	Sim+HW	Calibration + help	L3
A	Local Coding [135]	Code	S/M: code/history	HW	Task feedback	L3
A	MessyMem [5]	Mixed	M: scene graph	Sim+HW	Interaction	L3
A	MOKA [85]	Spatial	M/N: demos/policy	HW	Demo + distillation	L3
A	MOSAIC [149]	Mixed	N: actor/forecaster	HW	Offline training	L4(H)
A	MotorMind [67]	Mixed	State: task monitoring	Sim+HW	Task-time reasoning	L2
A	OntoPlan [106]	Skills	State	Sim	None established	L1
A	PhyAgentOS [88]	Skills	M/S: records/procedures	Sim+HW	Task feedback	L1
A	PhysCaP [82]	Code	S: transient code	HW	Task-time synthesis	L2
A	PiGeY [26]	Skills	State: execution state	Sim+HW	Task-time control	L1
A	PIVOT [107]	Commands	State: action search	Sim+HW	Task-time search	L1
A	PlanAR [32]	Skills	S: task plans	HW	Task-time planning	L2
A	PROC3S [19]	Code	S: code/search bounds	Sim+HW	Simulation feedback	L2
A	ProgPrompt [125]	Code	S: task plans	Sim+HW	Task-time synthesis	L2
A	REFLECT [92]	Skills	State: failure summary	Sim+HW	Task-time reflection	L2
A	ReKep [47]	Spatial	S: constraints	HW	Task-time synthesis	L2
A	ReMAC [182]	Skills	State	Sim	None established	L1
A	RoboDawn [31]	Commands	M/S: supplied demos	Sim+HW	External demos	L4(S)
A	RoboHarness [44]	Skills	M/H/E: cards/harness	Sim+HW	Rollout adaptation	L3
A	RoboPro [168]	Code	N: code generator	Sim+HW	Video2Code data	L3
A	RoboScript [13]	Code	S: task scripts	Sim+HW	Task-time synthesis	L2
A	RoboStream [48]	Mixed	M: rolling event graph	Sim+HW	Runtime accumulation	L2
A	RoboTool [173]	Code	S: task programs	Sim+HW	Task-time planning	L2
A	RoCo [103]	Mixed	S: subtask plans	Sim+HW	Dialogue + checks	L2
A	SafeHarness [171]	Mixed	H: fixed safety wrapper	Sim	Author design	L2
A	SayCan [50]	Skills	N: skills/affordances	HW	Offline BC/RL	L3
A	SayPlan [115]	Mixed	S: task plans	Sim+HW	Task-time planning	L2
A	Shared-KV BT [145]	Skills	State	Sim	None established	L1
A	ShowHarness [16]	Commands	N/M: model/demos	HW	Demo + training	L3
A	SimpleArm [194]	Skills	State: schema/memory	NR	Task-time synthesis	L2
A	Thea [153]	Mixed	M: session/tool memory	HW	Task consolidation	L2
A	VAP-TAMP [109]	Mixed	S/M: plans/scene graph	Sim+HW	Task-time planning	L2
A	VIA [39]	Mixed	H: fixed prompts	Sim+HW	Task-time control	L1
A	VLAs with Tools [65]	Mixed	N: tool-aligned policy	Sim	Offline training	L4(S)
A	VoxPoser [45]	Spatial	S/N: maps/dynamics	Sim+HW	Synthesis + interaction	L3
B. Capability Acquisition (91 records)						
B	AdaHVLA [138]	Mixed	H: coordination code	Sim+HW	Feedback revision	L4(S)
B	Agentic Robotics [180]	Indirect	N/E: policy/experiments	Sim	Experiment cycles	L3
B	Agentic RSR [71]	Integrated	M+N: policy/notes	Sim+HW	Sim curriculum	L3
B	AOR [61]	Code	S/H/M: code/memory	Sim	Explore + revise	L3
B	AquaBot [87]	Integrated	H: speed scales	HW	Surrogate search	L4(H)
B	Arcadia [28]	Integrated	E+N: data/policy	Sim+HW	Feedback + data	L3
B	ARCHITECT [10]	Code	S/M: skills/rules	HW	Human corrections	L4(H)
B	ASPIRE [96]	Code	S/M: repair library	Sim+HW	Debug + adaptation	L4(S)
B	AstraSim [37]	Mixed	S/M: code/experience	Sim+HW	Demo + revision	L4(S)
B	AtlasVA [151]	Integrated	M+N: memory/policy	Sim	Memory + PPO	L4(S)
B	AtomicVLA [187]	Integrated	N: skill experts	Sim+HW	Expert expansion	L3
B	AutoRT [1]	Skills	N: trained policy/data	HW	Collect + train	L3
B	CLS [62]	Code	S: policy archive	Sim	Demo + search	L4(S)
B	CodingTAMP [105]	Code	S/E: policies/tests	Sim	Test-driven search	L4(S)
B	COLADA [30]	Skills	S+N: adapters	Sim+HW	Dialogue + demos	L4(H)
B	Conquer [148]	Skills	M+N: skill bank	Sim+HW	Retrieval + RL	L4(S)
B	DAAG [110]	Integrated	E+N: data/reward	Sim	Video augmentation	L3
B	DexAgent [159]	Mixed	S/E/N: tools/policy	Sim+HW	Generate + train	L4(H)
B	DexFlywheel [205]	Integrated	E+N: data/policy	Sim+HW	Residual RL	L4(S)
B	DGAC [11]	Integrated	N: action/value	HW	Corrections	L3
B	DROC [184]	Code	M/S: rules/code	HW	Human feedback	L3
B	EmbodiedRSI [127]	Mixed	H + S + M + N	Sim+HW	Code/skill evolution	L4(B)
B	EmbodiedSmith [114]	Integrated	E+N: scenes/policy	Sim	Scene refinement	L3
B	EmbodiRSI [63]	Skills	N: policy/data	Sim+HW	Correct + recollect	L4(B)
B	ENCORE [57]	Code	S/M: programs/demos	Sim+HW	Demo + feedback	L4(S)
B	ENPIRE [165]	Mixed	N/E: policies/recipes	Sim+HW	Setup + experiments	L3
B	Eureka [100]	Indirect	E/N: reward/policy	Sim	Reward search + RL	L3
B	EvoHIL [188]	Integrated	N: actor/critic	HW	Human feedback	L3

Continued on the next page.

Table 8. Comparative overview (continued).

G.	Study	Agent interface	Capability artifact	Setting	Acquisition	Level
B. Capability Acquisition (91 records; continued)						
B	EvoVLA [91]	Integrated	M+N: policy	Sim+HW	Stages + PPO	L4(S)
B	GAP [14]	Mixed	S/H: skill graph	Sim+HW	Graph refinement	L3
B	GEAR [111]	Integrated	N: goal/distance	Sim	Human feedback	L3
B	Guava [86]	Mixed	N: actor weights	Sim+HW	Offline SFT/RL	L4(H)
B	HALP [112]	Skills	S+N: new skills	Sim+HW	Human demos	L3
B	HARBOR [72]	Indirect	E/N: tasks/recipes	Sim+HW	Generation + RL	L3
B	HarnessPAI [155]	Mixed	S/M/N: code/policy	Sim+HW	Repair + training	L3
B	HexaAnything [27]	Mixed	H/N: tools/model	Sim+HW	Revise + training	L3
B	InterMimicGen [193]	Integrated	E+N: data/tracker	Sim+HW	Iterative imitation	L3
B	KnowBody [95]	Mixed	M/H: body guidance	HW	Seed + interaction	L4(H)
B	LACY [38]	Integrated	N: action/language	Sim+HW	Consistency filtering	L4(S)
B	LMPC [79]	Spatial	N/E: teaching model	Sim+HW	Teaching + training	L4(B)
B	LOTUS [146]	Skills	S+N: skill bank	Sim+HW	Skill discovery	L3
B	LRLL [143]	Code	S/M/E: skills/tests	Sim+HW	Teacher + exploration	L4(S)
B	LYRA [104]	Code	M+S: code bank	Sim+HW	Human feedback	L3
B	MEDAL++ [123]	Integrated	N: forward/reset	Sim+HW	Autonomous RL	L4(S)
B	MobiAgent [84]	Skills	N: skills/data	Sim+HW	Curate + train	L3
B	MTRF [35]	Skills	N: task policies	Sim+HW	Reset-free RL	L3
B	OmniVIC [185]	Integrated	M+H: impedance	Sim+HW	Rollout retrieval	L4(S)
B	PhysEvo [141]	Code	S+H: tools/skills	Sim+HW	Meta-agent revision	L3
B	Plan-Seq-Learn [20]	Mixed	N: interaction policy	Sim	Plan-guided RL	L4(S)
B	PLD [164]	Integrated	N: generalist	Sim+HW	Residual distillation	L4(S)
B	Policy-Level RSI [36]	Integrated	N+H: experts/gate	Sim+HW	Risk-guided training	L4(B)
B	Q-Planning [29]	Skills	N: Q-function	Sim+HW	Deployment RL	L3
B	RACaP [73]	Skills	S/H/M: APIs/harness	Sim	Curriculum + revision	L4(S)
B	RAPID [90]	Mixed	S/E: strategies/scenes	Sim+HW	Simulation synthesis	L4(S)
B	RATs [186]	Code	S/M: helpers/lessons	Sim+HW	Play + consolidation	L4(B)
B	Real2Gym [118]	Code	E/S/M: sim/skills	Sim+HW	Reconstruct + practice	L3
B	Reflective-VLA [66]	Integrated	E+N: reward/policy	Sim	Reflection + RL	L3
B	ReLMM [134]	Skills	N: grasp/nav	Sim+HW	Autonomous RL	L3
B	Retaining Experiences [166]	Integrated	M+N: replay/policy	Sim+HW	Sequential RL	L4(S)
B	RHD [60]	Mixed	M: text playbooks	Sim+HW	Teacher refinement	L4(S)
B	RISE [174]	Integrated	N: world/policy	HW	Imagined rollouts	L3
B	Robo-COP [189]	Mixed	N + M	Sim+HW	Verified policy updates	L4(B)
B	RoboCat [9]	Integrated	N: policy/data	Sim+HW	Demos + self-data	L4(H)
B	RoboClaw [70]	Skills	N: policies/data	HW	Collect + training	L3
B	RoboFoundry [81]	Mixed	H/M/S: code/rules	Sim+HW	Feedback revision	L4(H)
B	RoboFuME [175]	Integrated	N: reward/policy	Sim+HW	Online RL	L4(H)
B	RoboHarn-Evo [6]	Skills	M: task/action rules	Sim+HW	Interaction + revise	L4(S)
B	RoboSkill [170]	Mixed	S/M: skill packages	Sim+HW	Explore + package	L4(S)
B	RSI [150]	Indirect	S/H/M: code/history	Sim	Explore + human repair	L3
B	RWLA [176]	Integrated	M+N: local policy	Sim	Retrieval + repair	L4(S)
B	SAS-Prompt [2]	Integrated	M+H: parameters	Sim+HW	Outcome summaries	L3
B	SECURE [119]	Skills	M: concept beliefs	Sim+HW	Human feedback	L3
B	SEES [75]	Integrated	N: family policies	Sim	Bottleneck RL	L4(S)
B	Self-Improving EFM [121]	Integrated	N: reward/policy	Sim+HW	Online RL	L3
B	SHAPER [152]	Skills	M/H: guidance/context	Sim	Feedback revision	L4(S)
B	SILVR [98]	Integrated	N: video/action	Sim+HW	Rollout fine-tuning	L3
B	SInViG [172]	Integrated	N: dialogue/grasp	HW	Generated dialogue	L4(H)
B	Skill-effect models [77]	Skills	N: effect model	Sim+HW	Sim transitions	L4(S)
B	SkillsCrafter [156]	Integrated	M+N: adapters	Sim+HW	Sequential demos	L4(H)
B	SkillWeaver [204]	Mixed	M/N: rules/policy	Sim+HW	Search + training	L3
B	SMART [4]	Integrated	E+N: configs/policy	Sim+HW	Generated demos	L4(H)
B	SOAR [203]	Skills	N: policy/data	HW	Explore + training	L3
B	SOE [55]	Integrated	N: latent policy	Sim+HW	Latent exploration	L3
B	Teach-and-Grow [108]	Code	S: skill blocks	Sim	Teacher traces	L4(S)
B	TidyBot [161]	Skills	M: preference rules	HW	Human examples	L2
B	Uni-Skill [169]	Skills	M+S: repository	Sim+HW	Offline demos	L4(S)
B	VaPRL [122]	Integrated	N: goal policy	Sim	Value curriculum	L4(S)
B	VideoAgent [133]	Integrated	N: video policy	Sim+HW	Online fine-tuning	L3
B	WAA [190]	Mixed	M/N: guidance/pilot	Sim	Demo + GUI correction	L4(S)
B	Zero2Skill [147]	Mixed	M/N: rules/policy	HW	Human + training	L4(H)
B	Zetta [21]	Skills	H: critic/recovery	Sim	Feedback revision	L4(S)
C. Mechanism-Focused Studies (18 records)						
C	AHA [23]	Indirect	N: diagnosis model	Sim+HW	Offline + feedback	L3
C	ARCHIE [142]	Skills	E+N: skills/rewards	Sim+HW	Reward code + RL	L3
C	ART [178]	Mixed	N: tool-use policy	Sim+HW	Offline training	L3
C	ECoT [183]	Integrated	N: reasoning policy	HW	Offline training	L4(H)
C	Embodied Skills [154]	Mixed	N: planner/skills	Sim+HW	Offline training	L3
C	FRAMES [144]	Mixed	H: fixed monitor	Sim	Fixed setup	L1
C	GVCCI [58]	Spatial	M+N: grounding	HW	Synthetic labels	L4(H)
C	HiVLA Study [40]	Mixed	M/N: memory/policy	Sim+HW	Summaries + training	L3
C	Leave No Trace [24]	Integrated	N: forward/reset	Sim	Joint RL	L3
C	Memory Anchors [22]	Integrated	M+N: replay/policy	Sim+HW	Replay curation	L4(H)
C	Object-model reuse [97]	Spatial	M: object models	HW	Reconstruction	L3
C	OneTwoVLA [83]	Integrated	N: reasoning policy	HW	Offline co-training	L4(H)
C	PACT [162]	Integrated	N: diffusion policy	Sim+HW	Safety curriculum	L3
C	PAINT [167]	Integrated	N: reversibility	Sim	Human labels + RL	L3
C	RETRIEVER [198]	Code	N: skill policies	Sim	Offline training	L3
C	SARM2 [15]	Integrated	N: reward/policy	HW	Autonomous data	L3
C	SRSA [34]	Skills	S+N: retrieval	Sim+HW	Prior skill transfer	L4(S)
C	VLA-Corrector [126]	Mixed	N: failure verifier	Sim+HW	Offline training	L4(H)

Codes. M: memory; S: skills/programs; H: harness/tools (H, T); N: neural parameters (θ); E: learning environments/rewards/recipes; State: episodic. HW: hardware; Sim: simulation; NR: not reported. L4(H/S/B): gain in hardware/simulation/both.

7. Discussion and Future Directions

The comparisons in Section 6 identify three linked design problems: locating the component responsible for a failure, making local updates compatible with subsequent execution, and sustaining acquisition under physical and human resource constraints. Retained artifacts already support gains in reported settings, and a bounded simulation stream combines repeated acquisition with transfer and earlier-task retention. The following directions address how such benefits could become more reliable across tasks and update cycles.

7.1 Methodological Reflections: Three Systemic Dilemmas

Observational Underdetermination in Physical Failure. In contact-rich manipulation, a failed rollout may reflect perception error, task decomposition, contact dynamics, calibration drift, or an inaccurate outcome verifier. A trace alone may not distinguish these explanations. Revising a prompt or policy without testing the proposed diagnosis can leave the failure unresolved or introduce new errors. The relevant question is whether a targeted intervention improves later outcomes under a matched evaluation.

Evaluator Gaming and Invariant Chaining Collapse. Candidate selection depends on the fidelity of the gate $\mathcal{G}(C_k, \widetilde{C}_{k+1}; \mathcal{V})$. Errors in geometric predicates or learned critics can admit updates that do not improve the required behavior. RSI [150] documents an accepted reachability outcome that leaves the next grasp infeasible. This motivates checking compatibility between a skill's terminal states and the next skill's initiation conditions, alongside independently assessed whole-task success. Evaluator exploitation is a risk to test, not an inevitable consequence of using a particular verifier.

The Conflation of Software Rollback and Physical Reset. Restoring a software configuration $\widetilde{C}_{k+1} \to C_k$ does not restore displaced objects or repair damaged hardware. Software rollback itself can require loading, dependency reconciliation, and verification. Physical recovery may require robot actions, fixtures, or human support. Reports should separate these operations and measure their time, interventions, and failed recoveries. An unreported labor cost remains unknown rather than evidence of concealed assistance.

7.2 Future Directions: Reliable and Sustained Capability Acquisition

Active Hypothesis-Disambiguating Probing. A testable direction is to let agents formulate competing failure hypotheses and choose interactions that distinguish them before revising C_k. PhysCaP [82] motivates contact-based information gathering. Future comparisons should evaluate targeted probing against untargeted retries under matched interaction and compute budgets, stating the expected diagnostic observation and whether the resulting update improves held-out manipulation.

Connecting Local Acceptance to Task Success. Candidate checks should examine whether a skill leaves states suitable for subsequent operations, alongside separate measurements of complete-task success. Comparing isolated skill checks with checks that include downstream requirements can test when local acceptance predicts useful composition. The selection rule $\mathcal{G}$ may remain fixed or itself be updated. For fixed rules, a key question is whether their judgments remain reliable as tasks and candidate distributions change. For revised rules, acceptance errors and changes in stringency should be tracked across versions, including harmful updates admitted on earlier tasks.

Autonomous Reset Operators and Labor Accounting. Reset and recovery policies can reduce externally supplied assistance during data collection [122, 123]. Their contribution should be

evaluated alongside setup, teaching, engineering, and deployment costs. We propose the following human labor ledger:

$$T_{\text{human}} = T_{\text{demo}} + T_{\text{setup}} + T_{\text{engineering}} + T_{\text{annotation}} + T_{\text{reset}} + T_{\text{rescue}} + T_{\text{eval}}. \tag{7}$$

Engineering includes harness repair, simulator construction, and training-pipeline configuration. Each person's time interval should be charged once to its primary purpose, with shared work allocated by a declared rule. Upfront reusable effort and recurring work should be reported separately, together with the accounting period and any amortization rule.

For L5, independent acquisition histories must jointly establish the specified gain, transfer, retention, and low-human-dependence criteria in the credited setting. The intervention threshold and measurement period should be stated before final evaluation, with interventions reported per episode, per successful task, and per acquisition cycle. Zero unscheduled assistance is a useful experimental target, but this review imposes neither a universal zero-human threshold nor an infinite-horizon convergence requirement.

7.3 Limitations of the Review

The findings of this survey are bounded by the following methodological constraints:

1. **Reported Evidence versus Latent Capabilities:** Systems are evaluated strictly against their substantiated empirical chains and public code repositories; undocumented validation protections or unverified latent performance claims cannot be credited retroactively.

2. **Conditioned Definition of Level 5:** No inspected chain satisfies the joint criteria under this review's intervention-accounting requirement. Accepting procedural automation and stated interaction budgets supports a narrower positive conclusion for repeatable simulated self-improvement. Changing the conjunction changes the composite label, while the component evidence remains available for other interpretations.

8. Conclusion

This review examines agent-mediated manipulation and capability acquisition across 162 primary studies. Retained programs, playbooks, and memory representations can improve skill selection, command formulation, and execution recovery using fixed underlying actors, while learned policies and verifiers adapt distinct system layers. However, aggregate benchmark gains can coexist with condition-specific losses, and local candidate successes do not ensure end-to-end task completion. The five-level framework organizes these findings, while component analysis separates demonstrated outcomes from joint criteria. When evaluated against procedural automation and stated interaction budgets, repeatable acquisition, transfer, and earlier-task retention occur jointly within bounded simulation streams. Under the stricter requirement of explicit human labor and physical reset accounting, independently repeated hardware self-improvement remains unresolved across the inspected corpus. Missing cost ledgers represent unmeasured boundaries rather than evidence of absent capability.

Advancing toward repeatable self-improvement requires an accountable experimental agenda: repeating acquisition across declared initial resources, isolating final evaluations from candidate selection, re-evaluating legacy tasks across update cycles, and publishing comprehensive human, reset, and compute ledgers. Grounding capability acquisition in these protocols connects retained artifacts to verifiable developmental autonomy in physical manipulation.

References

[1] Michael Ahn, Debidatta Dwibedi, Chelsea Finn, et al. AutoRT: Embodied foundation models for large scale orchestration of robotic agents. In *First Workshop on Vision-Language Models for Navigation and Manipulation at ICRA 2024*, 2024. URL https://openreview.net/forum?id=DYcCveNeR1.

[2] Heni Ben Amor, Laura Graesser, Atil Iscen, David D'Ambrosio, Saminda Abevruwan, Alex Bewley, Yifan Zhou, Kamalesh Kalirathinam, Swaroop Mishra, and Pannag Sanketi. Sas-prompt: large language models as numerical optimizers for robot self-improvement. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 10087–10094. IEEE, 2025.

[3] Huan ang Gao, Jiayi Geng, Wenyue Hua, et al. A survey of self-evolving agents: What, when, how, and where to evolve on the path to artificial super intelligence. *Transactions on Machine Learning Research*, 2026. ISSN 2835-8856. URL https://openreview.net/forum?id=CTr3bovS5F. Survey Certification.

[4] Jicong Ao, Shuhan Jiang, Yuling Zhong, Yanwen Liu, Yuhan Gao, Jiangyuan Zhao, Yang Zhang, Shiqiang Zhu, Chenjia Bai, and Xuelong Li. SMART: Zero-Shot Sim-to-Real Articulated Object Manipulation via Large-Scale Synthetic Pretraining. arXiv:2610.07652, 2026. URL https://arxiv.org/abs/2610.07652v1. Version v1.

[5] Anuva Banwasi, William Muckelroy III, Priya Sundaresan, Linfeng Zhao, Jeannette Bohg, and Cherie Ho. Messymem: Learning-from-doing memory for mobile manipulation. *arXiv preprint arXiv:2609.15976*, 2026.

[6] Shifeng Bao, Fanding Huang, Yihan Lin, Youhe Feng, Guanlin Li, Chen Zhao, Yang Li, Jiawei He, Cheng Chi, and Jing Zhang. Roboharn-evo: Evolving hierarchical physical knowledge for self-improving robotic manipulation. *arXiv preprint arXiv:2609.37583*, 2026.

[7] Kevin Black, Noah Brown, James Darpinian, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. In Joseph Lim, Shuran Song, and Hae-Won Park, editors, *Proceedings of The 9th Conference on Robot Learning*, volume 305 of *Proceedings of Machine Learning Research*, pages 17–40. PMLR, 27–30 Sep 2025. URL https://proceedings.mlr.press/v305/black25a.html.

[8] Kevin Black, Noah Brown, Danny Driess, et al. π_0: A Vision-Language-Action Flow Model for General Robot Control. In *Proceedings of Robotics: Science and Systems*, LosAngeles, CA, USA, June 2025. doi: 10.15607/RSS.2025.XXI.010.

[9] Konstantinos Bousmalis, Giulia Vezzani, Dushyant Rao, et al. Robocat: A self-improving generalist agent for robotic manipulation. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856. URL https://openreview.net/forum?id=vsCpILiWHu.

[10] Daphne Chen, Archit Ritesh Jain, Eric Goossen, Emma Romig, Michael Murray, Nick Walker, and Maya Cakmak. A few words go a long way: Language guided robot policy synthesis. *arXiv preprint arXiv:2607.23784*, 2026.

[11] Hanzhi Chen, Anran Zhang, Simon Schaefer, Kejia Chen, Shi Chen, Daniel Cremers, Oier Mees, and Stefan Leutenegger. Robot Self-Improvement via Human-Video Dynamics Models. arXiv:2606.21406, 2026. URL https://arxiv.org/abs/2606.21406v1. Version v1.

[12] Jiayi Chen, Wenxuan Song, Pengxiang Ding, Ziyang Zhou, Han Zhao, Feilong Tang, Donglin Wang, and Haoang Li. Unified Diffusion VLA: Vision-Language-Action Model via Joint Discrete Denoising Diffusion Process. arXiv:2511.01718, 2025. URL https://arxiv.org/abs/2511.01718v2. Version v2.

[13] Junting Chen, Yao Mu, Qiaojun Yu, Tianming Wei, Silang Wu, Zhecheng Yuan, Zhixuan Liang, Chao Yang, Kaipeng Zhang, Wenqi Shao, et al. Roboscript: Code generation for free-form manipulation tasks across real and simulation. *arXiv preprint arXiv:2402.14623*, 2024.

[14] Kaiyuan Chen, Shuangyu Xie, Letian Fu, et al. GaP: A Graph-as-Policy Multi-Agent Self-Learning Harness For Variational Automation Tasks, 2026. URL https://arxiv.org/abs/2607.05369v1.

[15] Qianzhong Chen, Hau Zheng, Justin Yu, Suning Huang, Jiankai Sun, Ken Goldberg, Chuan Wen, Pieter Abbeel, Yide Shentu, Philipp Wu, and Mac Schwager. SARM2: Multi-Task Stage Aware Reward Modeling for Self Improving Robotic Manipulation. arXiv:2606.10305, 2026. URL https://arxiv.org/abs/2606.10305v1. Version v1.

[16] Yanzhe Chen, Zechen Bai, Zhijun Cao, Wenzheng Zeng, Kevin Qinghong Lin, Yiqi Lin, Guoqiang Liang, Kevin Yuchen Ma, Qiming Huang, and Mike Zheng Shou. Show-harness: Just a vlm agent can play robots. *arXiv preprint arXiv:2609.10522*, 2026.

[17] Dongzhou Cheng, Taoran Yi, Ye Fang, Xingwu Zhang, Fan Feng, Yixuan Li, Gengxiong Zhuang, Rongze Wang, Shuai Yang, Wei Song, et al. In-context robot learning with vlm agents. *arXiv preprint arXiv:2609.19138*, 2026.

[18] Can Cui, Pengxiang Ding, Wenxuan Song, Shuanghao Bai, Xinyang Tong, Zirui Ge, Runze Suo, Wanqi Zhou, Yang Liu, Bofang Jia, Han Zhao, Siteng Huang, and Donglin Wang. OpenHelix: A Short Survey, Empirical Analysis, and Open-Source Dual-System VLA Model for Robotic Manipulation. arXiv:2505.03912, 2025. URL https://arxiv.org/abs/2505.03912v1. Version v1.

[19] Aidan Curtis, Nishanth Kumar, Jing Cao, Tomás Lozano-Pérez, and Leslie Pack Kaelbling. Trust the PRoc3s: Solving long-horizon robotics problems with LLMs and constraint satisfaction. In *8th Annual Conference on Robot Learning*, 2024. URL https://openreview.net/forum?id=r6ZhiVYriY.

[20] Murtaza Dalal, Tarun Chiruvolu, Devendra Chaplot, and Ruslan Salakhutdinov. Plan-seq-learn: Language model guided rl for solving long horizon robotics tasks. In *International Conference on Learning Representations*, volume 2024, pages 11087–11115, 2024.

[21] Xin Ding, Liang Mi, Mingzhe Huang, Zixuan Wang, Chao Zhang, Zixu Hao, Fu Chen, Xiangyu Li, Yikai Zheng, Yaoyu Guo, Weijun Wang, Kun Li, Hao Wu, Yunxin Liu, and Ting Cao. Zetta ζ: An efficient closed-loop embodied harness for self-evolving physical intelligence, 2026. URL https://arxiv.org/abs/2608.16590.

[22] Maximilian Du, Zhanyi Sun, Chen Xu, Paarth Shah, Masha Itkina, and Shuran Song. Memory Anchors for Continual Robot Learning. arXiv:2608.26545, 2026. URL https://arxiv.org/abs/2608.26545v1. Version v1.

[23] Jiafei Duan, Wilbert Pumacay, Nishanth Kumar, Yi Ru Wang, Shulin Tian, Wentao Yuan, Ranjay Krishna, Dieter Fox, Ajay Mandlekar, and Yijie Guo. AHA: A vision-language-model for detecting and reasoning over failures in robotic manipulation. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=JVkdSi7Ekg.

[24] Benjamin Eysenbach, Shixiang Gu, Julian Ibarz, and Sergey Levine. Leave no trace: Learning to reset for safe and autonomous reinforcement learning. In *International Conference on Learning Representations*, 2018. URL https://openreview.net/forum?id=S1vuO-bCW.

[25] Letian Fu, Justin Yu, Karim El-Refai, Ethan Kou, Haoru Xue, Huang Huang, Wenli Xiao, Li Fei-Fei, Guanya Shi, Jiajun Wu, S. Shankar Sastry, Yuke Zhu, Ken Goldberg, and Linxi Fan. Cap-x: A framework for benchmarking and improving coding agents for robot manipulation. In *Forty-third International Conference on Machine Learning*, 2026. URL https://openreview.net/forum?id=4JRO9plGAI.

[26] Liane Galanti, Dhruv Shah, and Tri Dao. Addressing the orchestration gap in generalist robots via physical agency. *arXiv preprint arXiv:2607.21725*, 2026.

[27] Hongcheng Gao, Jingjing Zhou, Zelin Zheng, Shijia Ge, Jay Zhu, Yazhe Wang, Jianshu Zeng, Xuan Shangguan, Di Wu, Lingyu He, Zhiqi Jia, Sihang Wu, and Xiao He. Self-Evolving Coding Agents: From Digital Programs to Physical-World Intelligence, 2026. URL https://arxiv.org/abs/2609.35432v1.

[28] Minghe Gao, Juncheng Li, Yuze Lin, Xuqi Liu, Jiaming Ji, Xiaoran Pan, Zihan Xu, Xian Li, Mingjie Li, Wei Ji, et al. Arcadia: Toward a full-lifecycle framework for embodied lifelong learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 1031–1040, 2026.

[29] Varun Giridhar, Anant Khandelwal, Jeremy A. Collins, Ignat Georgiev, and Animesh Garg. Beyond Imitation: Self-Improving Robot Policies via Off-Policy Q-Planning. arXiv:2608.21204, 2026. URL https://arxiv.org/abs/2608.21204v1. Version v1.

[30] Weiwei Gu, N Suresh K Kondepudi, Anmol Gupta, Lixiao Huang, and Nakul Gopalan. Continual robot skill and task learning via dialogue. In *ICRA 2025 Workshop on Foundation Models and Neuro-Symbolic AI for Robotics*, 2025. URL https://openreview.net/forum?id=BPK2OaKVdy.

[31] Meng-Hao Guo, Zhe-Han Mo, Jia-Jun Wang, Yi Zhang, Kejin Wang, Yi-Xuan Deng, Jia-Peng Zhang, Yongming Rao, and Shi-Min Hu. Transferring the intelligence of vlms to robotic control. *arXiv preprint arXiv:2609.22966*, 2026.

[32] Pengyuan Guo, Zhonghao Mai, Zhengtong Xu, Kaidi Zhang, Quan Khanh Luu, Heng Zhang, Zichen Miao, Arash Ajoudani, Zachary Kingston, Qiang Qiu, et al. Planar: Planning-language-grounded agentic reasoning for robot manipulation. *arXiv preprint arXiv:2602.01662*, 2026.

[33] Yanjiang Guo, Yen-Jen Wang, Lihan Zha, and Jianyu Chen. Doremi: Grounding language model by detecting and recovering from plan-execution misalignment. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 12124–12131. IEEE, 2024.

[34] Yijie Guo, Bingjie Tang, Iretiayo Akinola, Dieter Fox, Abhishek Gupta, and Yashraj Narang. Srsa: Skill retrieval and adaptation for robotic assembly tasks. In *International Conference on Learning Representations*, volume 2025, pages 17082–17111, 2025.

[35] Abhishek Gupta, Justin Yu, Tony Z Zhao, Vikash Kumar, Aaron Rovinsky, Kelvin Xu, Thomas Devlin, and Sergey Levine. Reset-free reinforcement learning via multi-task learning: Learning dexterous manipulation behaviors without human intervention. In *2021 IEEE international conference on robotics and automation (ICRA)*, pages 6664–6671. IEEE, 2021.

[36] Linxuan He, Yuying Tian, Lingxiang Fan, Jiaqi Pi, Yinqiao Lu, Shang Su, Mengkai Shi, and Shuo Feng. Policy-Level Recursive Self-Improvement for Embodied AI with a Criticality World Model. arXiv:2607.28251, 2026. URL https://arxiv.org/abs/2607.28251v2. Version v2.

[37] Sida He, Lingxi Xie, Yunning Cao, Pengfei Chen, Kaiwen Duan, Jiannan Ge, Xinyue Huo, Jiacheng Shao, and Qi Tian. Robot Manipulation with GPT-6-Astra: Body Knowledge, Experience Reuse, Emergent Skills, and Sim2Real Transfer, 2026. URL https://arxiv.org/abs/2609.31770v1.

[38] Youngjin Hong, Houjian Yu, Mingen Li, and Changhyun Choi. Lacy: A vision-language model-based language-action cycle for self-improving robotic manipulation. In *2026 IEEE International Conference on Robotics and Automation (ICRA)*, pages 10339–10346. IEEE, 2026.

[39] Hengyuan Hu, Priya Sundaresan, Jensen Gao, and Dorsa Sadigh. VIA: Visual Interface Agent for Robot Control, 2026. URL https://arxiv.org/abs/2607.11119v1.

[40] Jiaheng Hu, Mohit Shridhar, Caden Lu, Dhruv Shah, Hao-Tien Lewis Chiang, Jie Tan, and Annie Xie. What Matters in Orchestrating Robot Policies: A Systematic Study of Hierarchical VLA Agents, 2026. URL https://arxiv.org/abs/2606.10267v2.

[41] Yutong Hu, Fengjiao Chen, Xuezhi Cao, and Renaud Detry. 2am: Grounding agent-side memory as guidance for steerable action models in long-horizon manipulation. *arXiv preprint arXiv:2609.11308*, 2026.

[42] Haojian Huang, Zexi Li, Junhao Guo, Yehang Zhang, Wenxuan Peng, Bohan Zhou, Weilin Ruan, Leyi Wu, Chenxu Wang, Jianchong Su, et al. In-context learning for robots: Methods and applications. *arXiv preprint arXiv:2609.36012*, 2026.

[43] Jiakai Huang, Zhongbo Wu, Siyu Xu, Zheng Zhang, Zihan Wang, Shan You, Chang Xu, and Tao Huang. Foresight without seeing: Latent futures for world action models. *arXiv preprint arXiv:2608.11605*, 2026.

[44] Jinbang Huang, Yuanzhao Hu, Zhiyuan Li, Ran Qi, Yixin Xiao, Zhanguang Zhang, Mark Coates, Tongtong Cao, and Yingxue Zhang. Roboharness: Memory-driven orchestration of heterogeneous robot policies for long-horizon planning. *arXiv preprint arXiv:2607.18060*, 2026.

[45] Wenlong Huang, Chen Wang, Ruohan Zhang, Yunzhu Li, Jiajun Wu, and Li Fei-Fei. Voxposer: Composable 3d value maps for robotic manipulation with language models. In Jie Tan, Marc Toussaint, and Kourosh Darvish, editors, *Proceedings of The 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 540–562. PMLR, 06–09 Nov 2023. URL https://proceedings.mlr.press/v229/huang23b.html.

[46] Wenlong Huang, Fei Xia, Ted Xiao, et al. Inner monologue: Embodied reasoning through planning with language models. In Karen Liu, Dana Kulic, and Jeff Ichnowski, editors, *Proceedings of The 6th Conference on Robot Learning*, volume 205 of *Proceedings of Machine Learning Research*, pages 1769–1782. PMLR, 14–18 Dec 2023. URL https://proceedings.mlr.press/v205/huang23c.html.

[47] Wenlong Huang, Chen Wang, Yunzhu Li, Ruohan Zhang, and Li Fei-Fei. Rekep: Spatio-temporal reasoning of relational keypoint constraints for robotic manipulation. In Pulkit Agrawal, Oliver Kroemer, and Wolfram Burgard, editors, *Proceedings of The 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 4573–4602. PMLR, 06–09 Nov 2025. URL https://proceedings.mlr.press/v270/huang25g.html.

[48] Yuzhi Huang, Jie Wu, Weijue Bu, Ziyi Xiong, Gaoyang Jiang, Ye Li, Kangye Ji, Shuzhao Xie, Yue Huang, Chenglei Wu, et al. Robostream: Weaving s patio-t emporal rea soning with m emory in vision-language models for robo tics. In *European Conference on Computer Vision*, pages 37–56. Springer, 2026.

[49] Dongjie Huo, Haoyun Liu, Guoqing Liu, Dekang Qi, Zhiming Sun, Maoguo Gao, Jianxin He, Yandan Yang, Xinyuan Chang, Feng Xiong, Xing Wei, Zhiheng Ma, and Mu Xu. ABot-Claw: A Foundation for Persistent, Cooperative, and Self-Evolving Robotic Agents, 2026. URL https://arxiv.org/abs/2604.10096v2.

[50] brian ichter, Anthony Brohan, Yevgen Chebotar, et al. Do as i can, not as i say: Grounding language in robotic affordances. In Karen Liu, Dana Kulic, and Jeff Ichnowski, editors, *Proceedings of The 6th Conference on Robot Learning*, volume 205 of *Proceedings of Machine Learning Research*, pages 287–318. PMLR, 14–18 Dec 2023. URL https://proceedings.mlr.press/v205/ichter23a.html.

[51] Yuheng Ji, Huajie Tan, Jiayu Shi, Xiaoshuai Hao, Yuan Zhang, Hengyuan Zhang, Pengwei Wang, Mengdi Zhao, Yao Mu, Pengju An, et al. Robobrain: A unified brain model for robotic manipulation from abstract to concrete. In *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1724–1734. IEEE, 2025.

[52] Yuheng Ji, Cheng Chi, Huajie Tan, Yuting Zhao, Huaihai Lyu, Enshen Zhou, Pengwei Wang, Zhongyuan Wang, Xiaolong Zheng, and Shanghang Zhang. From ill-posed to well-posed: A decomposition framework for reliable robot learning with foundation models. *IEEE Transactions on Robotics*, 2026.

[53] Yuheng Ji, Yuyang Liu, Huajie Tan, Xuchuan Huang, Fanding Huang, Yijie Xu, Cheng Chi, Yuting Zhao, Huaihai Lyu, Peterson Co, et al. Prm-as-a-judge: A dense evaluation paradigm for fine-grained robotic auditing. *arXiv preprint arXiv:2603.21669*, 2026.

[54] Mengzhao Jia, Yang Lin, Xixin Zhang, Zhihan Zhang, Xiaobai Liu, and Meng Jiang. Agent as Policy for Robotic Manipulation, 2026. URL https://arxiv.org/abs/2609.12541v3.

[55] Yang Jin, Jun Lv, Han Xue, Wendi Chen, Chuan Wen, and Cewu Lu. Soe: Sample-efficient robot policy self-improvement via on-manifold exploration. In *2026 IEEE International Conference on Robotics and Automation (ICRA)*, pages 1062–1069. IEEE, 2026.

[56] Frank Joublin, Antonello Ceravola, Pavel Smirnov, Felix Ocker, Joerg Deigmoeller, Anna Belardinelli, Chao Wang, Stephan Hasler, Daniel Tanneberg, and Michael Gienger. Copal: corrective planning of robot actions with large language models. In *2024 ieee international conference on robotics and automation (ICRA)*, pages 8664–8670. IEEE, 2024.

[57] Yifan Kang, Zihan Wang, Zhiwen Fan, and Bangya Liu. Encore: Few-Shot Agentic Discovery of Manipulation Strategies, 2026. URL https://arxiv.org/abs/2609.37359v1.

[58] Junghyun Kim, Gi-Cheon Kang, Jaein Kim, Suyeon Shin, and Byoung-Tak Zhang. Gvcci: Lifelong learning of visual grounding for language-guided robotic manipulation. In *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 952–959. IEEE, 2023.

[59] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, et al. Openvla: An open-source vision-language-action model. In Pulkit Agrawal, Oliver Kroemer, and Wolfram Burgard, editors, *Proceedings of The 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 2679–2713. PMLR, 06–09 Nov 2025. URL https://proceedings.mlr.press/v270/kim25c.html.

[60] Seungyeon Kim, Junhoo Lee, Minkyu Kim, Baekseung Kim, and Nojun Kwak. Recursive harness distillation across agents for robot manipulation. *arXiv preprint arXiv:2609.33378*, 2026.

[61] Vaishak Kumar. Act-observe-rewrite: Multimodal coding agents as in-context policy learners for robot manipulation. *arXiv preprint arXiv:2603.04466*, 2026.

[62] So Kuroki and Yujin Tang. Learning and transferring closed-loop robot software. *arXiv preprint arXiv:2609.19906*, 2026.

[63] Haoran Lang, Haotao Lu, Shiyu Sang, Haoyang Luo, Guo Chen, Qun Li, Jingyi Yu, Ye Shi, and Jingya Wang. Embodirsi: Recursive self-improvement for data-efficient robot adaptation, 2026. URL https://arxiv.org/abs/2609.38905v3. Preprint; inspected 2609.38905v3.

[64] Huashuo Lei, Wenxuan Song, Huarui Zhang, Jieyuan Pei, Jiayi Chen, Haodong Yan, Han Zhao, Pengxiang Ding, Zhipeng Zhang, Lida Huang, Donglin Wang, Yan Wang, and Haoang Li. RoboMemArena: A Comprehensive and Challenging Robotic Memory Benchmark. arXiv:2605.10921, 2026. URL https://arxiv.org/abs/2605.10921v1. Version v1.

[65] Zixing Lei, Changxing Liu, Yichen Xiong, Minhao Xiong, Yuanzhuo Ding, Zhipeng Zhang, Weixin Li, and Siheng Chen. Towards Long-horizon Embodied Agents with Tool-Aligned Vision-Language-Action Models, 2026. URL https://arxiv.org/abs/2605.13119v1.

[66] Baicheng Li, Dong Wu, Zike Yan, Xinchen Liu, Lusong Li, Zecui Zeng, and Hongbin Zha. Reflection-Based Task Adaptation for Self-Improving VLA. arXiv:2510.12710, 2025. URL https://arxiv.org/abs/2510.12710v3. Version v3.

[67] Bingxuan Li, Siqi Song, Yizhuo Wu, Jiarui Yao, Tong Zhang, and Huan Zhang. MotorMind: Scaffolding General Vision Language Models for Zero-Shot Robot Manipulation, 2026. URL https://arxiv.org/abs/2609.38078v1.

[68] Fuhao Li, Wenxuan Song, Han Zhao, Jingbo Wang, Pengxiang Ding, Donglin Wang, Long Zeng, and Haoang Li. Spatial Forcing: Implicit Spatial Representation Alignment for Vision-language-action Model. arXiv:2510.12276, 2025. URL https://arxiv.org/abs/2510.12276v2. Version v2.

[69] Jialiang Li, Yuhan Wang, Haojun Li, Gaojing Zhang, Yangtian Ye, Qipeng Liu, Haotian Liang, and Wenzhao Lian. Practice Makes Policies: Bootstrapping and Consolidating Robotic Capabilities from Zero Human Demonstrations. arXiv:2607.26809, 2026. URL https://arxiv.org/abs/2607.26809v1. Version v1.

[70] Ruiying Li, Yunlang Zhou, YuYao Zhu, Kylin Chen, Sukai Wang, Kongtao Hu, Minhui Yu, Bowen Jiang, Zhan Su, Jiayao Ma, et al. Roboclaw: An agentic framework for scalable long-horizon robotic tasks. In *European conference on computer vision*, pages 304–320. Springer, 2026.

[71] Yihan Li, Yating Feng, Shengjiu Sun, Jianing Chen, Hao Ren, Bowen Yang, Weisheng Xu, Qiwei Wu, Hui Cheng, and Renjing Xu. Agentic RSR: Real-to-Sim-to-Real through Scene Reconstruction and Execution-Grounded Robot Policies. arXiv:2610.10479, 2026. URL https://arxiv.org/abs/2610.10479v1. Version v1.

[72] Zechu Li, Yufeng Jin, Xiaoyang Liu, Puze Liu, Vignesh Prasad, Carlo D'Eramo, and Georgia Chalvatzaki. HARBOR: A Harness Framework for Agentic Robot Reinforcement Learning, 2026. URL https://arxiv.org/abs/2606.08610v1.

[73] Zexi Li, Yehang Zhang, Haojian Huang, Bohan Zhou, Wenqian Li, Chenxu Wang, Yifan Chang, Yangkai Wei, Tianyi Zhang, Ying-Cong Chen, et al. Racap: Agentic reasoning, acting, and coding as policies for evolvable robot learning. *arXiv preprint arXiv:2609.29394*, 2026.

[74] Zexi Li, Yehang Zhang, Wenqian Li, Haojian Huang, Chenxu Wang, Shiyuan Deng, Yangkai Wei, Tianyi Zhang, Binghui Xie, Bohan Zhou, et al. Robo-harness k1: Harnessing robot-use agents via perception augmentation. *arXiv preprint arXiv:2609.29389*, 2026.

[75] Ziwen Li, Hanlue Zhang, Zhenyang Ren, Tianyu Huang, Runqi Lin, Haoyu Wang, Zhengqing Gao, Yandong Guo, Fakhri Karray, Tongliang Liu, Chris Russell, and Mingming Gong. SEES: A Self-Evolving Embodied System via Failure-Guided VLA Policy Adaptation. arXiv:2609.32698, 2026. URL https://arxiv.org/abs/2609.32698v1. Version v1.

[76] Anthony Liang, Yigit Korkmaz, Jiahui Zhang, Minyoung Hwang, Abrar Anwar, Sidhant Kaushik, Aditya Shah, Alex S Huang, Luke Zettlemoyer, Dieter Fox, et al. Robometer: Scaling general-purpose robotic reward models via trajectory comparisons. *arXiv preprint arXiv:2603.02115*, 2026.

[77] Jacky Liang, Mohit Sharma, Alex LaGrassa, Shivam Vats, Saumya Saxena, and Oliver Kroemer. Search-based task planning with learned skill effect models for lifelong robotic manipulation. In *2022 International Conference on Robotics and Automation (ICRA)*, pages 6351–6357. IEEE, 2022.

[78] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. In *2023 IEEE International conference on robotics and automation (ICRA)*, pages 9493–9500. IEEE, 2023.

[79] Jacky Liang, Fei Xia, Wenhao Yu, et al. Learning to learn faster from human feedback with language model predictive control. In *Robotics: Science and Systems*, 2024. URL https://doi.org/10.15607/RSS.2024.XX.125.

[80] Jiaqi Liang, Yue Chen, Qize Yu, Yan Shen, Haipeng Zhang, Hao Dong, and Ruihai Wu. A3D: Adaptive affordance assembly with dual-arm manipulation. *Proceedings of the AAAI Conference on Artificial Intelligence*, 40 (22):18425–18433, 2026. doi: 10.1609/aaai.v40i22.38907. URL https://ojs.aaai.org/index.php/AAAI/article/view/38907.

[81] Jingsong Liang, Shuhao Liao, Shizhe Zhang, Diyuan Hou, Yuxin Cai, Xinjian Deng, Chengyang He, Wenhui Huang, Runjia Tan, Zhidong Wang, et al. Robofoundry: System-as-policy evolution for self-learning embodied agents. *arXiv preprint arXiv:2609.32862*, 2026.

[82] Chen-Yu Lin, Jing-Wen Chen, Hsueh-En Chang, Hung-An Chen, Sheng-Hsun Chang, Chi-Pin Huang, Fu-En Yang, Min-Hung Chen, Yi-Ting Chen, Yu-Chiang Frank Wang, et al. Physcap: Grounding code-as-policy agent with physics-informed exploration. *arXiv preprint arXiv:2608.21031*, 2026.

[83] Fanqi Lin, Ruiqian Nai, Yingdong Hu, Jiacheng You, Junming Zhao, and Yang Gao. Onetwovla: A unified vision-language-action model with adaptive reasoning. *arXiv preprint arXiv:2505.11917*, 2025.

[84] Chenzhi Liu, Yue Zhang, Jiehong Lin, Jianan Wang, Bo Wang, Zhongrui Wang, and Xiaojuan Qi. Mobiagent: Dual-loop recursive policy self-improvement for long-horizon mobile manipulation, 2026. URL https://arxiv.

org/abs/2610.03476v1. Preprint; inspected 2610.03476v1; CoRL 2026 acceptance reported in arXiv comment.

[85] Fangchen Liu, Kuan Fang, Pieter Abbeel, and Sergey Levine. MOKA: Open-vocabulary robotic manipulation through mark-based visual prompting. In *First Workshop on Vision-Language Models for Navigation and Manipulation at ICRA 2024*, 2024. URL https://openreview.net/forum?id=K8eoYUofbQ.

[86] Haowen Liu, Xirui Li, Shaoxiong Yao, Peng Shi, Tianyi Zhou, Jia-Bin Huang, Furong Huang, and Jiayuan Mao. Guava: An effective and universal harness for embodied manipulation. *arXiv preprint arXiv:2606.18363*, 2026.

[87] Ruoshi Liu, Huy Ha, Mengxue Hou, Shuran Song, and Carl Vondrick. Self-improving autonomous underwater manipulation. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 16915–16922. IEEE, 2025.

[88] Yang Liu, Weixing Chen, Xinshuai Song, Tao Pu, Siwen Mo, Yongjie Bai, Zihao Chen, Qianran Sun, Liruo Zhong, Ying Shen, et al. Phyagentos: A self-evolving operating system for embodied agents with decoupled cognitive planning and physical execution. *arXiv preprint arXiv:2607.16636*, 2026.

[89] Yuyang Liu, Yanqing Shen, Ruike Chen, Jifan Zhao, Yuxuan Tian, Yichi Zhang, Tianfeng Long, Zixuan Yin, Yipu Wang, Ziheng Qin, et al. Prm-as-a-judge 1.5: A toolkit for robot process assessment. *arXiv preprint arXiv:2608.14284*, 2026.

[90] Yuyao Liu, Jiayuan Mao, David Hsu, Leslie Pack Kaelbling, and Tomás Lozano-Pérez. RAPID: Robot Agentic Programming from Demonstrations, 2026. URL https://arxiv.org/abs/2609.30249v1.

[91] Zeting Liu, Zida Yang, Zeyu Zhang, and Hao Tang. Evovla: Self-evolving vision-language-action model. In *European Conference on Computer Vision*, pages 241–260. Springer, 2026.

[92] Zeyi Liu, Arpit Bahety, and Shuran Song. Reflect: Summarizing robot experiences for failure explanation and correction. In Jie Tan, Marc Toussaint, and Kourosh Darvish, editors, *Proceedings of The 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 3468–3484. PMLR, 06–09 Nov 2023. URL https://proceedings.mlr.press/v229/liu23g.html.

[93] Zhen Liu, Xinyu Ning, Zhe Hu, Xinxin Xie, Weize Li, Zhipeng Tang, Chongyu Wang, Zejun Yang, Hanlin Wang, Yitong Liu, et al. Goal2skill: Long-horizon manipulation with adaptive planning and reflection. *arXiv preprint arXiv:2604.13942*, 2026.

[94] Yayu Long, Kewei Chen, Long Jin, and Mingsheng Shang. Drae: Dynamic retrieval-augmented expert networks for lifelong learning and task adaptation in robotics. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 23098–23141, 2025.

[95] Zeyu Lou, Yanhong Zeng, Yong Wang, and Chenyang Si. Know Your Body: A Harness for Direct and Self-Improving Robot Control with VLMs, 2026. URL https://arxiv.org/abs/2609.28530v1.

[96] Runyu Lu, Yubo Wu, Ethan Kou, Letian Fu, Wenli Xiao, Ajay Mandlekar, Yinzhen Xu, Guanya Shi, Ken Goldberg, Ang Chen, et al. Aspire: Agentic/skills discovery for robotics. *arXiv preprint arXiv:2607.00272*, 2026.

[97] Shiyang Lu, Rui Wang, Yinglong Miao, Chaitanya Mitash, and Kostas Bekris. Online object model reconstruction and reuse for lifelong improvement of robot manipulation. In *2022 International Conference on Robotics and Automation (ICRA)*, pages 1540–1546. IEEE, 2022.

[98] Calvin Luo, Zilai Zeng, Mingxi Jia, Yilun Du, and Chen Sun. Self-improving loops for visual robotic planning. In *International Conference on Learning Representations*, volume 2026, pages 126885–126913, 2026.

[99] Jindi Lv, Hao Li, Jie Li, Fankun Kong, Yang Wang, Pengfei Yi, Yifei Nie, Xiaofeng Wang, Zheng Zhu, Chaojun Ni, et al. Viva: A video-generative value model for robot reinforcement learning. *arXiv preprint arXiv:2604.08168*, 2026.

[100] Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Eureka: Human-level reward design via coding large language models. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=IEduRUO55F.

[101] Yecheng Jason Ma, Joey Hejna, Chuyuan Fu, Dhruv Shah, Jacky Liang, Zhuo Xu, Sean Kirmani, Peng Xu, Danny Driess, Ted Xiao, et al. Vision language models are in-context value learners. In *International Conference on Learning Representations*, volume 2025, pages 33984–34009, 2025.

[102] Yueen Ma, Zixing Song, Yuzheng Zhuang, Jianye Hao, and Irwin King. A survey on vision–language–action models for embodied ai. *IEEE Transactions on Neural Networks and Learning Systems*, 2026.

[103] Zhao Mandi, Shreeya Jain, and Shuran Song. Roco: Dialectic multi-robot collaboration with large language models. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 286–299. IEEE, 2024.

[104] Yuan Meng, Zhenguo Sun, Max Fest, Xiangtong Yao, Xukun Li, Zhenshan Bing, and Alois Knoll. Robotic Ultra-Long-Horizon Manipulation Skills via Human-guided Lifelong Code Generation. arXiv:2509.18597, 2025. URL https://arxiv.org/abs/2509.18597v3. Version v3.

[105] Matteo Merler, Bowen Li, Josh Roy, Yichao Liang, Qianwei Wang, Yixuan Huang, and Tom Silver. Coding Agents for Generalized Task and Motion Planning Problems, 2026. URL https://arxiv.org/abs/2609.30233v1.

[106] Hyeongwoo Nam, Woongje Cho, Juwon Kim, and Jongeun Choi. OntoPlan: An Ontology-Grounded Scene Representation and Agentic Framework for Scalable Robot Task Planning. arXiv:2610.07649, 2026. URL https://arxiv.org/abs/2610.07649v1. Version v1; NeurIPS 2026 (reported in arXiv metadata).

[107] Soroush Nasiriany, Fei Xia, Wenhao Yu, et al. PIVOT: Iterative visual prompting elicits actionable knowledge for VLMs. In *Forty-first International Conference on Machine Learning*, 2024. URL https://openreview.net/forum?id=051jaf8MQy.

[108] Chang Nie, Zhe Liu, and Hesheng Wang. Teach and Grow: An Agent-Centered Architecture for General Robot Learning. arXiv:2608.17209, 2026. URL https://arxiv.org/abs/2608.17209v3. Version v3.

[109] Austine Oloo, Zainab Altaweel, Yohei Hayamizu, Peiqi Liu, Yan Ding, Saeid Amiri, Hao Yang, Andy Kaminski, Chad Esselink, Chris Paxton, Xiaohan Zhang, and Shiqi Zhang. Robot Planning and Situation Handling with Active Perception, 2026. URL https://arxiv.org/abs/2604.26988v1.

[110] Norman Di Palo, Leonard Hasenclever, Jan Humplik, and Arunkumar Byravan. Diffusion augmented agents: A framework for efficient exploration and transfer learning. In Vincenzo Lomonaco, Stefano Melacci, Tinne Tuytelaars, Sarath Chandar, and Razvan Pascanu, editors, *Proceedings of The 3rd Conference on Lifelong Learning Agents*, volume 274 of *Proceedings of Machine Learning Research*, pages 268–284. PMLR, 29 Jul–01 Aug 2025. URL https://proceedings.mlr.press/v274/palo25a.html.

[111] Max Balsells I Pamies, Marcel Torne Villasevil, Zihan Wang, Samedh Desai, Pulkit Agrawal, and Abhishek Gupta. Autonomous robotic reinforcement learning with asynchronous human feedback. In *7th Annual Conference on Robot Learning*, 2023.

[112] Meenal Parakh, Alisha Fong, Anthony Simeonov, Tao Chen, Abhishek Gupta, and Pulkit Agrawal. Lifelong robot learning with human assisted language planners. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 523–529. IEEE, 2024.

[113] Jiaqi Peng, Xiqian Yu, Delin Feng, Yuqiang Yang, Wenzhe Cai, Jing Xiong, Ganlin Yang, Jinliang Zheng, Jiafei Cao, Xueyuan Wei, et al. Cortex: A bidirectionally aligned embodied agent framework for long-horizon manipulation. *arXiv preprint arXiv:2607.05377*, 2026.

[114] Yikai Qin, Yifei Deng, Mingjian Liang, Wenxuan Song, Zepeng Lin, Zhiyi Jiang, Jiajun Fu, Qiao Sun, Huashuo Lei, Xicheng Gong, Jiayi Chen, Han Zhao, Shuanghao Bai, Pengxiang Ding, Pengwei Wang, and Haoang Li. EmbodiedSmith: Scaling Embodied Data through Recursive Self-Improvement Flywheel in Simulation. arXiv:2610.07969, 2026. URL https://arxiv.org/abs/2610.07969v1. Version v1.

[115] Krishan Rana, Jesse Haviland, Sourav Garg, Jad Abou-Chakra, Ian Reid, and Niko Suenderhauf. Sayplan: Grounding large language models using 3d scene graphs for scalable robot task planning. In Jie Tan, Marc Toussaint, and Kourosh Darvish, editors, *Proceedings of The 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 23–72. PMLR, 06–09 Nov 2023. URL https://proceedings.mlr.press/v229/rana23a.html.

[116] Emmanuel K Raptis, Athanasios Ch Kapoutsis, and Elias B Kosmatopoulos. Agentic llm-based robotic systems for real-world applications: a review on their agenticness and ethics. *Frontiers in Robotics and AI*, 12:1605405, 2025.

[117] Allen Z. Ren, Anushri Dixit, Alexandra Bodrova, et al. Robots that ask for help: Uncertainty alignment for large language model planners. In Jie Tan, Marc Toussaint, and Kourosh Darvish, editors, *Proceedings of The 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 661–682. PMLR, 06–09 Nov 2023. URL https://proceedings.mlr.press/v229/ren23a.html.

[118] Kerui Ren, Yingxiang Xu, Kaiwen Song, Linning Xu, Bo Dai, Mulin Yu, and Tao Lu. Real2gym: Building gyms from videos, bringing skills to robots. *arXiv preprint arXiv:2609.37089*, 2026.

[119] Rimvydas Rubavicius, Peter David Fagan, Alex Lascarides, and Subramanian Ramamoorthy. SECURE: Semantics-aware Embodied Conversation under Unawareness for Lifelong Robot Learning. arXiv:2409.17755, 2024. URL https://arxiv.org/abs/2409.17755v3. Version v3; CoLLAs 2025 (reported in arXiv metadata).

[120] Sahar Salimpour, Lei Fu, Kajetan Rachwał, Pascal Bertrand, Kevin O'Sullivan, Robert Jakob, Farhad Keramat, Leonardo Militano, Giovanni Toffetti, Harry Edelman, et al. Towards embodied agentic ai: Review and classification of llm-and vlm-driven robot autonomy and interaction. *arXiv preprint arXiv:2508.05294*, 2025.

[121] Seyed Kamyar Seyed Ghasemipour, Ayzaan Wahid, Jonathan Tompson, Pannag Sanketi, and Igor Mordatch. Self-improving embodied foundation models. *Advances in Neural Information Processing Systems*, 38:112651–112686, 2026.

[122] Archit Sharma, Abhishek Gupta, Sergey Levine, Karol Hausman, and Chelsea Finn. Autonomous reinforcement learning via subgoal curricula. *Advances in Neural Information Processing Systems*, 34:18474–18486, 2021.

[123] Archit Sharma, Ahmed M Ahmed, Rehaan Ahmad, and Chelsea Finn. Self-improving robots: End-to-end autonomous visuomotor reinforcement learning. In *7th Annual Conference on Robot Learning*, 2023. URL https://openreview.net/forum?id=ApxLUk8U-l.

[124] Lucy Xiaoyang Shi, brian ichter, Michael Robert Equi, Liyiming Ke, Karl Pertsch, Quan Vuong, James Tanner, Anna Walling, Haohuan Wang, Niccolo Fusai, Adrian Li-Bell, Danny Driess, Lachy Groom, Sergey Levine, and Chelsea Finn. Hi robot: Open-ended instruction following with hierarchical vision-language-action models. In *Forty-second International Conference on Machine Learning*, 2025. URL https://openreview.net/forum?id=lNVHg9npif.

[125] Ishika Singh, Valts Blukis, Arsalan Mousavian, Ankit Goyal, Danfei Xu, Jonathan Tremblay, Dieter Fox, Jesse Thomason, and Animesh Garg. Progprompt: Generating situated robot task plans using large language models. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 11523–11530, 2023. doi: 10.1109/ICRA48891.2023.10161317.

[126] Chang Song, Bin Qian, Yan Feng, and Zhijie Song. VLA-Corrector: Stage-Aware Observable State Understanding for Prompt-Based Closed-Loop Recovery of Vision-Language-Action Policies, 2026. URL https://arxiv.org/abs/2609.06508v1.

[127] Python Song, Zhixuan Liang, Kelsey Fu, Mengdi Wang, Junfeng Yang, and Shilong Liu. Embodiedrsi: Active continual robot learning through hypothesis-guided co-evolution, 2026. URL https://arxiv.org/abs/2610.10498v1. Preprint; inspected 2610.10498v1.

[128] Wenxuan Song, Jiayi Chen, Pengxiang Ding, Han Zhao, Wei Zhao, Zhide Zhong, Zongyuan Ge, Zhijun Li, Donglin Wang, Jun Ma, Lujia Wang, and Haoang Li. PD-VLA: Accelerating Vision-Language-Action Model Integrated with Action Chunking via Parallel Decoding. arXiv:2503.02310, 2025. URL https://arxiv.org/abs/2503.02310v2. Version v2.

[129] Wenxuan Song, Jiayi Chen, Wenxue Li, Xu He, Han Zhao, Can Cui, Pengxiang Ding, Shiyan Su, Feilong Tang, Donglin Wang, Xuelian Cheng, Zongyuan Ge, Xinhu Zheng, Zhe Liu, Hesheng Wang, and Haoang Li. RationalVLA: A Rational Vision-Language-Action Model with Dual System. arXiv:2506.10826, 2025. URL https://arxiv.org/abs/2506.10826v2. Version v2.

[130] Wenxuan Song, Ziyang Zhou, Han Zhao, Jiayi Chen, Pengxiang Ding, Haodong Yan, Yuxin Huang, Feilong Tang, Donglin Wang, and Haoang Li. ReconVLA: Reconstructive Vision-Language-Action Model as Effective Robot Perceiver. arXiv:2508.10333, 2025. URL https://arxiv.org/abs/2508.10333v1. Version v1.

[131] Wenxuan Song, Jiayi Chen, Jingbo Wang, Shuai Zhou, Xicheng Gong, Zehua Fan, Ziyang Zhou, Junwu E, Haodong Yan, Fuhao Li, Qize Yu, Xu Huang, Pengwei Wang, Wen Chen, Shunbo Zhou, and Haoang Li. UniWAM: Unified World-Action Model. arXiv:2610.02054, 2026. URL https://arxiv.org/abs/2610.02054v3. Version v3.

[132] Wenxuan Song, Han Zhao, Fuhao Li, Ziyang Zhou, Xi Wang, Jing Lyu, Pengxiang Ding, Yan Wang, Donglin Wang, and Haoang Li. CapVector: Learning Transferable Capability Vectors in Parametric Space for Vision-Language-Action Models. arXiv:2605.10903, 2026. URL https://arxiv.org/abs/2605.10903v1. Version v1.

[133] Achint Soni, Sreyas Venkataraman, Abhranil Chandra, Sebastian Fischmeister, Percy Liang, Bo Dai, and Sherry Yang. Videoagent: Self-improving video generation for embodied planning. In *NeurIPS 2025 Workshop on Bridging Language, Agent, and World Models for Reasoning and Planning*, 2025.

[134] Charles Sun, Jedrzej Orbik, Coline Manon Devin, Brian H Yang, Abhishek Gupta, Glen Berseth, and Sergey Levine. Fully autonomous real-world reinforcement learning with applications to mobile manipulation. In *Conference on Robot Learning*, pages 308–319. PMLR, 2022.

[135] Raman Talwar, Elias Nijs, Andreas Verleysen, and Francis wyffels. Generalizing Manipulation Skills with a Local Coding Agent, 2026. URL https://arxiv.org/abs/2609.26499v1.

[136] Huajie Tan, Sixiang Chen, Yijie Xu, Zixiao Wang, Yuheng Ji, Cheng Chi, Yaoxu Lyu, Zhongxia Zhao, Xiansheng Chen, Peterson Co, et al. Robo-dopamine: General process reward modeling for high-precision robotic manipulation. *arXiv preprint arXiv:2512.23703*, 2025.

[137] Huajie Tan, Cheng Chi, Xiansheng Chen, Yuheng Ji, Zhongxia Zhao, Xiaoshuai Hao, Yaoxu Lyu, Mingyu Cao, Junkai Zhao, Huaihai Lyu, et al. Roboos-next: A unified memory-based framework for lifelong, scalable, and robust multi-robot collaboration. *arXiv preprint arXiv:2510.26536*, 2025.

[138] Junyi Tang, Jie Peng, Zezhen Ding, Yuan Shen, and Tianlong Chen. AdaHVLA: Adaptive Harnesses for Long-Horizon Vision-Language-Action Execution, 2026. URL https://arxiv.org/abs/2609.29204v1.

[139] BAAI RoboBrain Team, Mingyu Cao, Huajie Tan, Yuheng Ji, Xiansheng Chen, Minglan Lin, Zhiyu Li, Zhou Cao, Pengwei Wang, Enshen Zhou, et al. Robobrain 2.0 technical report. *arXiv preprint arXiv:2507.02029*, 2025.

[140] Kairos Team, Fei Wang, Shan You, Qiming Zhang, Tao Huang, Zuoyi Fu, Zhisheng Zheng, Yunlong Xi, Feng Lv, Xiaoming Wu, Zeyu Liu, Cong Wan, Pu Li, Ruiqing Yang, Xiaoou Li, Wei Wang, Kangkang Zhu, Yuwei Zhang, Shi Fu, Zheng Zhang, Xiaoning Wu, Xuzeng Fan, Dacheng Tao, and Xiaogang Wang. Kairos: A regret-aware native world-action model stack for physical ai, 2026. URL https://arxiv.org/abs/2606.16533.

[141] Wenqing Tian, Zeyu Zhang, Zhaocheng Liu, Fengwei Liu, Qiang Liu, and Liang Wang. PhysEvo: Astra Can Act, Let It. arXiv:2610.08995, 2026. URL https://arxiv.org/abs/2610.08995v1. Version v1.

[142] Niccolò Turcato, Matteo Iovino, Aris Synodinos, Alberto Dalla Libera, Ruggero Carli, and Pietro Falco. Towards autonomous reinforcement learning for real-world robotic manipulation with large language models. *IEEE Robotics and Automation Letters*, 10(9):8850–8857, 2025.

[143] Georgios Tziafas and Hamidreza Kasaei. Lifelong robot library learning: Bootstrapping composable and generalizable skills for embodied control with language models. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 515–522. IEEE, 2024.

[144] Ajay Vikram Periasami, Xinyuan Luo, Haoyu Li, and Xianyi Cheng. Frames: Failure recovery and monitoring of embodied skills for humanoid loco-manipulation. *arXiv e-prints*, pages arXiv–2609, 2026.

[145] Naoki Wake and Justin Wagle. SharedKV-BT: Node-Local Typed Decisions for Behavior-Tree Agents. arXiv:2610.07327, 2026. URL https://arxiv.org/abs/2610.07327v1. Version v1.

[146] Weikang Wan, Yifeng Zhu, Rutav Shah, and Yuke Zhu. Lotus: Continual imitation learning for robot manipulation through unsupervised skill discovery. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 537–544. IEEE, 2024.

[147] Boyuan Wang, Zhenyuan Zhang, Zhiqin Yang, Peijun Gu, Shuya Wang, Xiaofeng Wang, Xianghui Ze, Yifan Chang, Guosheng Zhao, Jiangnan Shao, Guan Huang, Hengyu Liu, Yonggang Zhang, Wei Xue, Chunyuan Guan, Chenglin Pu, Yike Guo, Xingang Wang, and Zheng Zhu. Zero2Skill: Bootstrapping Robot Skills through Autonomous Data Collection, Training, and Deployment, 2026. URL https://arxiv.org/abs/2607.14047v3.

[148] Daoqing Wang, Yuchen Xiao, Weixuan Huang, Zhilong Zhang, Shenghua Wan, Meng Li, Lei Yuan, and Yang Yu. Continual Quadruped Robots Coordination via Semantic Skill Discovery. arXiv:2606.08102, 2026. URL https://arxiv.org/abs/2606.08102v2. Version v2.

[149] Huaxiaoyue Wang, Kushal Kedia, Juntao Ren, et al. MOSAIC: Modular foundation models for assistive and interactive cooking. In *8th Annual Conference on Robot Learning*, 2024. URL https://openreview.net/forum?id=dUo6j3YURS.

[150] Jiaming Wang. What stops recursive self-improvement in robotics? lessons from 123 rounds of agentic skill discovery. *arXiv preprint arXiv:2609.31760*, 2026.

[151] Pan Wang, Yihao Hu, Xiujin Liu, Jingchu Yang, Hang Wang, and Zhihao Wen. AtlasVA: Self-Evolving Visual Skill Memory for Teacher-Free VLM Agents. arXiv:2605.17933, 2026. URL https://arxiv.org/abs/2605.17933v1. Version v1.

[152] Peidong Wang, Zhiming Ma, Ying Chang, Xufang Luo, Yiqun Zhang, Zihan Wang, Xiaocui Yang, Shi Feng, Yuqing Yang, and Dongsheng Li. Self-evolving embodied agents via skill-harness evolution. *arXiv preprint arXiv:2608.11350*, 2026.

[153] Qi Wang, Tianyi Wang, Chengyang Li, Shikun Ban, Yurun Chen, Yizhong Ge, Jason Qin, Chengtai Li, and Wentao Zhu. Towards the Harness of Embodied Agents, 2026. URL https://arxiv.org/abs/2608.11246v1.

[154] Wei Wang, Wenqiao Zhang, Yutong Lin, Yuqian Yuan, Tianwei Lin, Jinhao Mao, Zhenxuan Fan, Mingjian Gao, Yang Dai, Wentong Li, Zheqi Lv, Zheng Dong, Yingjie Niu, Jiaqi Zhu, Jun Xiao, Chao Li, and Yueting Zhuang. EmbodiedSkills: A Unified Framework for Orchestrating, Training, and Deploying VLA Agents, 2026. URL https://arxiv.org/abs/2609.01281v1.

[155] Xin Wang, Wenhao Wu, Menghao Zhang, Zhi Wang, Kun Shao, Jian Luan, Yang Li, Qing Li, Shangding Gu, Huichi Zhou, Shuqing Shi, Fei Ni, Shuo Lu, Weicheng Meng, Kang Li, Jin Wu, Kang Zhao, Shangmin Guo, Gen Li, Yongqiang Tang, Zhizhong Zhang, Yuan Xie, and Heng Qu. HarnessPAI: An Evolving Harness for Physical AI, 2026. URL https://arxiv.org/abs/2609.29166v1.

[156] Xudong Wang, Zebin Han, Zhiyu Liu, Gan Li, Jiahua Dong, Baichen Liu, Lianqing Liu, and Zhi Han. Lifelong language-conditioned robotic manipulation learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 18629–18637, 2026.

[157] Yi Wang, Xinchen Li, Pengwei Xie, Pu Yang, Buqing Nie, Yunuo Cai, Qinglin Zhang, Chendi Qu, Jeffrey Wu, Jianheng Song, et al. Learning while deploying: Fleet-scale reinforcement learning for generalist robot policies. *arXiv preprint arXiv:2605.00416*, 2026.

[158] Yihao Wang, Yuheng Ji, Mingyu Cao, Yanqing Shen, Runze Xiao, Huaihai Lyu, Senwei Xie, Euan Liu, Klara Tian, Tianfeng Long, et al. Orca: The world is in your mind. *arXiv preprint arXiv:2606.30534*, 2026.

[159] Youhui Wang, Yunzhu Li, Li Fei-Fei, Jiajun Wu, and Huang Huang. DexAgent: An Agentic Human2Sim2Robot Framework for Dexterous Manipulation with Self-Evolving Tool Library, 2026. URL https://arxiv.org/abs/2609.35318v1.

[160] Zhihao Wang, Jianxiong Li, Yu Cui, Yuan Gao, Xianyuan Zhan, Junzhi Yu, and Xiao Ma. World value models for robotic manipulation. *arXiv preprint arXiv:2606.24742*, 2026.

[161] Jimmy Wu, Rika Antonova, Adam Kan, Marion Lepert, Andy Zeng, Shuran Song, Jeannette Bohg, Szymon Rusinkiewicz, and Thomas Funkhouser. Tidybot: Personalized robot assistance with large language models. *Autonomous Robots*, 47(8):1087–1102, 2023.

[162] Lingxuan Wu, Zijian Zhu, Lizhong Wang, Chengyang Ying, Huayu Chen, Xiao Yang, Fangming Liu, and Jun Zhu. PACT: Self-Evolving Physical Safety Alignment for Diffusion Policies in Embodied Manipulation. arXiv:2606.08414, 2026. URL https://arxiv.org/abs/2606.08414v1. Version v1.

[163] Tianshu Wu, Xiangqi Kong, Yue Chen, Qize Yu, Hang Ye, Jia Li, Yizhou Wang, and Hao Dong. SUGAR: A scalable human-video-driven generalizable humanoid loco-manipulation learning framework, 2026. URL https://arxiv.org/abs/2605.20373.

[164] Wenli Xiao, Haotian Lin, Andy Peng, Haoru Xue, Tairan He, Zhengyi Luo, Yuqi Xie, Fengyuan Hu, Jim Fan, Guanya Shi, et al. Self-improving vision-language-action models with data generation via residual rl. In *International Conference on Learning Representations*, volume 2026, pages 13209–13236, 2026.

[165] Wenli Xiao, Jia Xie, Tonghe Zhang, Haotian Lin, Letian Fu, Haoru Xue, Jalen Lu, Yi Yang, Cunxi Dai, Zi Wang, et al. Enpire: Agentic robot policy self-improvement in the real world. *arXiv preprint arXiv:2606.19980*, 2026.

[166] Annie Xie and Chelsea Finn. Lifelong robotic reinforcement learning by retaining experiences. In *Conference on Lifelong Learning Agents*, pages 838–855. PMLR, 2022.

[167] Annie Xie, Fahim Tajwar, Archit Sharma, and Chelsea Finn. When to ask for help: Proactive interventions in autonomous reinforcement learning. *Advances in Neural Information Processing Systems*, 35:16918–16930, 2022.

[168] Senwei Xie, Hongyu Wang, Zhanqi Xiao, Ruiping Wang, and Xilin Chen. Robotic programmer: Video instructed policy code generation for robotic manipulation. In *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 14923–14930. IEEE, 2025.

[169] Senwei Xie, Yuntian Zhang, Ruiping Wang, and Xilin Chen. Uni-Skill: Building Self-Evolving Skill Repository for Generalizable Robotic Manipulation. arXiv:2603.02623, 2026. URL https://arxiv.org/abs/2603.02623v1. Version v1; ICRA 2026 (reported in arXiv metadata).

[170] Sicheng Xie, Yitong Chen, Haidong Cao, Shunlin Lu, Zuxuan Wu, and Yu-Gang Jiang. Explore, Execute, Evolve: A Skill Acquisition and Reuse Loop for Embodied Agents, 2026. URL https://arxiv.org/abs/2609.37810v1.

[171] Bingxin Xu, Yuzhang Shang, Zhen Dong, and Emilio Ferrara. Coding Agents with an Obstacle-Aware Harness for Safe Robot Manipulation, 2026. URL https://arxiv.org/abs/2609.20822v1.

[172] Jie Xu, Hanbo Zhang, Xinghang Li, Huaping Liu, Xuguang Lan, and Tao Kong. SInViG: A Self-Evolving Interactive Visual Agent for Human-Robot Interaction. arXiv:2402.11792, 2024. URL https://arxiv.org/abs/2402.11792v2. Version v2.

[173] Mengdi Xu, Peide Huang, Wenhao Yu, Shiqi Liu, Xilun Zhang, Yaru Niu, Tingnan Zhang, Fei Xia, Jie Tan, and Ding Zhao. Creative robot tool use with large language models. *arXiv preprint arXiv:2310.13065*, 2023.

[174] Jiazhi Yang, Kunyang Lin, Jinwei Li, Wencong Zhang, Tianwei Lin, Longyan Wu, Zhizhong Su, Hao Zhao, Ya-Qin Zhang, Li Chen, Ping Luo, Xiangyu Yue, and Hongyang Li. RISE: Self-Improving Robot Policy with Compositional World Model. arXiv:2602.11075, 2026. URL https://arxiv.org/abs/2602.11075v2. Version v2; RSS 2026 (reported in arXiv metadata).

[175] Jingyun Yang, Max Sobol Mark, Brandon Vu, Archit Sharma, Jeannette Bohg, and Chelsea Finn. Robot fine-tuning made easy: Pre-training rewards and policies for autonomous real-world reinforcement learning. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 4804–4811. IEEE, 2024.

[176] Pengzhi Yang, Xinyu Wang, Ruipeng Zhang, Cong Wang, Frans A. Oliehoek, and Jens Kober. Task-agnostic Lifelong Robot Learning with Retrieval-based Weighted Local Adaptation. arXiv:2410.02995, 2024. URL https://arxiv.org/abs/2410.02995v4. Version v4.

[177] Zhejian Yang, Yongchao Chen, Xueyang Zhou, Jiangyue Yan, Dingjie Song, Yinuo Liu, Yuting Li, Yu Zhang, Pan Zhou, Hechang Chen, et al. Agentic robot: A brain-inspired framework for vision-language-action models in embodied agents. *arXiv preprint arXiv:2505.23450*, 2025.

[178] Ding Yi, Yanzhao Yu, Xili Dai, Xianbiao Qi, Peiwen Sun, Xueqian Wang, Xiangyu Yue, and Jianan Wang. Evolve vision-language-action model into an agent with on-the-fly tool-use. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 1346–1357, 2026.

[179] Jiadi You, Qize Yu, Yue Chen, Minghong Cai, Zhide Zhong, Yuran Wang, Bowen Ping, Jiaqi Liang, Zhenhao Shen, Haodong Yan, Yinchuan Li, Ruihai Wu, Xiaojuan Qi, and Yingcong Chen. AffordanceWAM: Affordance-aware joint world-action modeling for robot manipulation, 2026. URL https://arxiv.org/abs/2609.22332.

[180] Hang Yu. You Don't Need To Stay in The Loop: An Agentic Robotics Loop for Robot-Policy Improvement, 2026. URL https://arxiv.org/abs/2608.07555v1.

[181] Qize Yu, Jiadi You, Yuran Wang, Jiaqi Liang, Bowen Ping, Yang Tian, Yue Chen, Minghong Cai, Zeying Gong, Ruihai Wu, Yinchuan Li, Junwei Liang, and Yingcong Chen. AffordanceVLA: A vision-language-action model empowering action generation through affordance-aware understanding, 2026. URL https://arxiv.org/abs/2606.06155.

[182] Puzhen Yuan, Angyuan Ma, Yunchao Yao, Huaxiu Yao, Masayoshi Tomizuka, and Mingyu Ding. Remac: Self-reflective and self-evolving multi-agent collaboration for long-horizon robot manipulation. *Pattern Recognition*, page 114831, 2026.

[183] Michał Zawalski, William Chen, Karl Pertsch, Oier Mees, Chelsea Finn, and Sergey Levine. Robotic control via embodied chain-of-thought reasoning. In Pulkit Agrawal, Oliver Kroemer, and Wolfram Burgard, editors, *Proceedings of The 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 3157–3181. PMLR, 06–09 Nov 2025. URL https://proceedings.mlr.press/v270/zawalski25a.html.

[184] Lihan Zha, Yuchen Cui, Li-Heng Lin, Minae Kwon, Montserrat Gonzalez Arenas, Andy Zeng, Fei Xia, and Dorsa Sadigh. Distilling and retrieving generalizable knowledge for robot manipulation via language corrections. In *2024 IEEE international conference on robotics and automation (ICRA)*, pages 15172–15179. IEEE, 2024.

[185] Heng Zhang, Wei-Hsing Huang, Gokhan Solak, and Arash Ajoudani. Omnivic: A self-improving variable impedance controller with vision-language in-context learning for safe robotic manipulation. *arXiv preprint arXiv:2510.17150*, 2025.

[186] Junyi Zhang, Jiaxin Ge, Hanjun Yoo, Letian Fu, Zihan Yang, Yaowei Liu, Raj Saravanan, Shaofeng Yin, Justin Yu, Dantong Niu, et al. Playful agentic robot learning. *arXiv preprint arXiv:2606.19419*, 2026.

[187] Likui Zhang, Tao Tang, Zhihao Zhan, Xiuwei Chen, Zisheng Chen, Jianhua Han, Jiangtong Zhu, Pei Xu, Hang Xu, Hefeng Wu, Liang Lin, and Xiaodan Liang. AtomicVLA: Unlocking the potential of atomic skill learning in robots. In *Conference on Computer Vision and Pattern Recognition 2026*, 2026. URL https://openreview.net/forum?id=DGxNEtiUGr.

[188] Shuoqin Zhang, Tongtong Cheng, Xiru Gao, Jinzhuo Peng, Bin Zheng, Jiahao Tu, Ke Wang, Jia Pan, Zhe Hu, and Kai Liu. EvoHIL: Self-Evolving Reward and Flow-Matched Policy Optimization for Robust Human-in-the-Loop Reinforcement Learning. arXiv:2608.03872, 2026. URL https://arxiv.org/abs/2608.03872v1. Version v1.

[189] Xilun Zhang, Maggie Wang, Erik Bauer, Hong-Xing Yu, Huang Huang, Jiajun Wu, and Marco Pavone. Co-evolving robot orchestrators and policies through deployment, 2026. URL https://arxiv.org/abs/2610.09228v1. Preprint; inspected 2610.09228v1.

[190] Yehang Zhang, Haojian Huang, Yifan Chang, Jianchong Su, Bohan Zhou, Yingjie Xu, Wosong Chen, Tianhao Zhou, Chenxu Wang, Tianyi Zhang, et al. World action agent: Harnessing vlms for robot manipulation via world action rehearsal. *arXiv preprint arXiv:2609.29964*, 2026.

[191] Yixian Zhang, Huanming Zhang, Feng Gao, Xiao Li, Zhihao Liu, Chunyang Zhu, Jiaxing Qiu, Yuchen Yan, Jiyuan Liu, Wenhao Tang, et al. Harness vla: Steering frozen vlas into reliable manipulation primitives via memory-guided agents. *arXiv preprint arXiv:2607.08448*, 2026.

[192] Yongheng Zhang, Ziang Liu, Jiaxuan Zhu, Shuai Wang, Xiangqi Chen, Haojing Huang, Jiayi Kuang, Siyu Chen, Ao Shen, Hao Wu, et al. From chatbot to digital colleague: The paradigm shift toward persistent autonomous ai. *arXiv preprint arXiv:2606.14502*, 2026.

[193] Yucheng Zhang, Sirui Xu, Jinhong Li, Liuyu Bian, Anatulya Nandi, Derek Zhang, Xiangchen Liu, Xueting Li, Umar Iqbal, Yu-Xiong Wang, and Liang-Yan Gui. InterMimicGen: Scaling Humanoid Loco-Manipulation through Self-Evolving Motion Imitation. arXiv:2610.06850, 2026. URL https://arxiv.org/abs/2610.06850v1. Version v1.

[194] Yuyou Zhang, Yunbei Zhang, Miao Li, Janet Wang, Zijian Jin, Shilong Liu, and Ding Zhao. Simple agentic memory for generalist robot policies. *arXiv preprint arXiv:2609.36595*, 2026.

[195] Zheng Zhang, Zhixiang Guo, Yuan Si, Siyuan Liang, Shunyu Liu, Weibo Gao, Song Wang, Leszek Rutkowski, Giuseppe Valenzise, Ming-Hsuan Yang, Lin William Cong, M. Jamal Deen, Sally Cripps, and Dacheng Tao. A Survey of World Model Benchmarks. Preprints.org, 2026. URL https://www.preprints.org/manuscript/202609.0665.

[196] Zheng Zhang, Xiaoyang Tang, Shuaiqi Cheng, Jiakai Huang, Zhongbo Wu, Ziyi Zhang, Yuan Si, Zeyu Liu, Weibo Gao, Shunyu Liu, Shan You, Fei Wang, Xiaogang Wang, and Dacheng Tao. Worldfailbench: Do world models understand policy failures?, 2026. URL https://arxiv.org/abs/2610.33565.

[197] Han Zhao, Jiaxuan Zhang, Wenxuan Song, Pengxiang Ding, and Donglin Wang. VLA^2: Empowering Vision-Language-Action Models with an Agentic Framework for Unseen Concept Manipulation. arXiv:2510.14902, 2025. URL https://arxiv.org/abs/2510.14902v1. Version v1.

[198] Linfeng Zhao, Haojie Huang, Jiayuan Mao, Weiyu Liu, Mykel Kochenderfer, and Lawson L. S. Wong. Retriever: Composing the Perception-Reasoning-Action Loop for Long-Horizon Manipulation, 2026. URL https://arxiv.org/abs/2607.17213v2.

[199] Zixiang Zhao, Yansong Feng, Yang Yang, Chaoyu Wang, Haoran Xiao, Hui Zhang, Chuang Cheng, and Jianjun Ma. Commitflow: Semantic commitment verification and local correction for long-horizon robot manipulation vla execution. *arXiv preprint arXiv:2609.21908*, 2026.

[200] Zhide Zhong, Haodong Yan, Junfeng Li, Xiangchen Liu, Xin Gong, Tianran Zhang, Wenxuan Song, Jiayi Chen, Xinhu Zheng, Hesheng Wang, and Haoang Li. FlowVLA: Visual Chain of Thought-based Motion Reasoning for Vision-Language-Action Models. arXiv:2508.18269, 2025. URL https://arxiv.org/abs/2508.18269v3. Version v3.

[201] Bohan Zhou, Xingbei Chen, Emily Huang, Weilin Ruan, Haojian Huang, Yehang Zhang, Zexi Li, Wenqian Li, Qize Yu, Zetian Song, Leyi Wu, Jinghao Li, Mingxuan Song, Xinrun Xu, Zongyang Qiu, Yangkai Wei, Tianyi Zhang, Kaiwen Zhou, Yinchuan Li, and James Cheng. Roboaware: Learning to coordinate embodied skills from counterfactual outcomes, 2026. URL https://arxiv.org/abs/2610.11480.

[202] Enshen Zhou, Qi Su, Cheng Chi, Zhizheng Zhang, Zhongyuan Wang, Tiejun Huang, Lu Sheng, and He Wang. Code-as-monitor: Constraint-aware visual programming for reactive and proactive robotic failure detection. In *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 6919–6929. IEEE, 2025.

[203] Zhiyuan Zhou, Pranav Atreya, Abraham Lee, Homer Rich Walke, Oier Mees, and Sergey Levine. Autonomous improvement of instruction following skills via foundation models. In *8th Annual Conference on Robot Learning*, 2024. URL https://openreview.net/forum?id=8Ar8b00GJC.

[204] He Zhu, Lusen Zhao, Kwan Man Cheng, Su Li, and Katerina Fragkiadaki. Skillweaver: Agentic exploration over neural interaction skills for scalable robot data generation. *arXiv preprint arXiv:2609.36171*, 2026.

[205] Kefei Zhu, Fengshuo Bai, YuanHao Xiang, Yishuai Cai, Xinglin Chen, Ruochong Li, Xingtao Wang, Hao Dong, Yaodong Yang, Xiaopeng Fan, et al. Dexflywheel: A scalable and self-improving data generation framework for dexterous manipulation. *Advances in Neural Information Processing Systems*, 38:4236–4265, 2026.

[206] Brianna Zitkovich, Tianhe Yu, Sichun Xu, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In Jie Tan, Marc Toussaint, and Kourosh Darvish, editors, *Proceedings of The 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 2165–2183. PMLR, 06–09 Nov 2023. URL https://proceedings.mlr.press/v229/zitkovich23a.html.